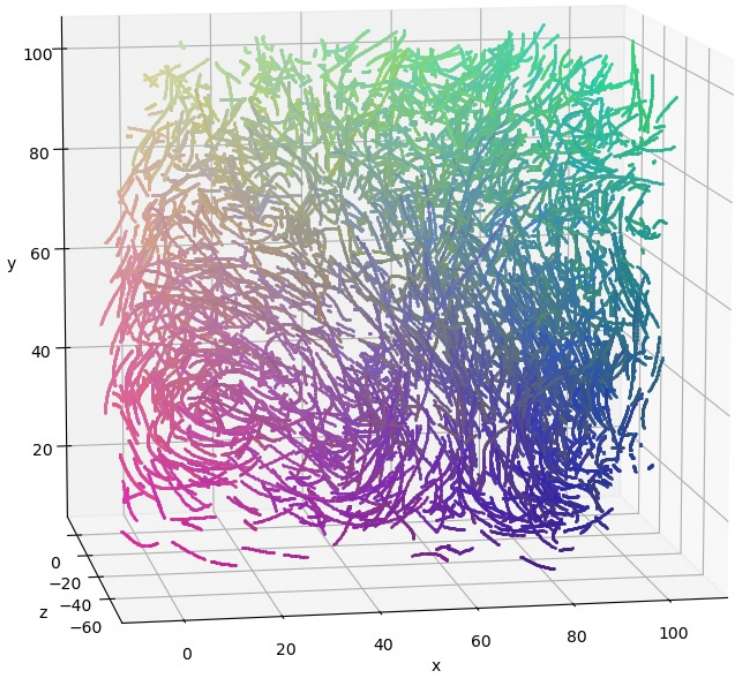
User Manual



Version: 1.3.15
Last updated: 1st September 2026
Github repository: https://github.com/ronshnapp/MyPTV
Get help & interact with our community: https://github.com/ronshnapp/MyPTV/discussions
Contact the developers: ronshnapp@gmail.com

Contents

1 Introduction

1.1 3D-PTV principles

3D-PTV is a method used to measure the trajectories of particles in 3D space. It utilizes the principles of stereoscopic vision to reconstruct 3D positions of particles from images taken from several orientations. A scheme of a typical 3D-PTV experiment using a four-camera system is shown in Fig. 1a. The main components comprising the 3D-PTV method are the co-linearity condition and the 3D model. According to the co-linearity condition, the position of a point in 3D space can be identified from the crossing of two or more straight lines in 3D space. Therefore, if we have a system of cameras that all of them see a common particle, then the particles' position can be estimated by finding the point in which rays of light connecting the particle with the different cameras cross each other. To perform this task we use a 3D model – a mathematical representation of the way in which cameras "see" the world. Thus, the 3D model is a function that can convert 3D points into pixel coordinates (η, ζ in Fig. 1b) and pixel coordinates back into rays of light [1, 2]. For example, A commonly used 3D model is the pin-hole camera model, in which each camera is associated with a position and orientation in 3D space (O' and θ in Fig. 1b). MyPTV has two 3D models that the user can choose from and they are described below.

Figure 1: (a) A schematics of a 3D-PTV experiment. (b) A schematic description of the 3d model, the pin-hole camera model.

Once the experiment, namely data acquisition, is done, there are six intrinsic post-processing steps to follow in order to complete the analysis and obtain the particles' trajectories. The six steps are outlined in Fig. 2. In the camera calibration step, we use images of known calibration targets to fit our physical cameras to the 3D model. In particle segmentation, we use image analysis to obtain the particles' image space coordinates (η, ζ). In the particle matching step, we use the co-linearity condition to identify which particle images in the different cameras correspond to the same physical particle, and triangulate their positions. In particle tracking, we connect the positions of particles in 3D space to form trajectories. In data conditioning, we might use smoothing and re-tracking algorithms to enhance our data quality according to some physical heuristics. Lastly, we can analyze the data to obtain information on the physics of the particles we are studying. The MyPTV package is meant to handle the first five of these steps.

The sections that follow outline both how MyPTV can be used by end users and also details on the code used internally to comply with the 3D-PTV method.

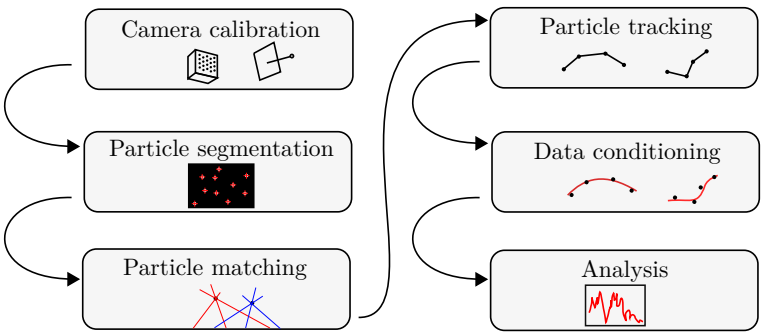


Figure 2: Basic steps in the analysis of PTV raw data into particle trajectories and scientific output. The first five steps are handled by MyPYV.

1.2 MyPTV usage and structure

MyPTV is used to conduct the post-processing steps in which the experimental raw data (images) are transformed into particles' trajectories. The five steps include camera calibration, image segmentation, particle matching, particle tracking, and, data conditioning which is divided into smoothing and stitching of broken trajectories. This section outlines how these steps are to be followed and how MyPTV is structured to facilitate them. Section 3 gives instructions on how to efficiently use MyPTV.

At the current stage, MyPTV does not include a graphical user interface. Instead, the package includes a script that can be used to efficiently and conveniently run the various post-processing steps. Knowledge of Python is not required to operate MyPTV.

The internal structure of MyPTV is made of separate "modules", each of which deals with a particular step out of the list described above. Each module consists of a Python script file that includes one or several classes. Detailed information on each of these classes is given in Sections 4–10, and the code itself includes documentation to a level appropriate for development. In addition to the source code, MyPTV includes a *workflow* script that is used to operate the software in an organized fashion. Thus, MyPTV can be used either by using the workflow script or by directly evoking classes from the source code.

2 MyPTVs 3D models

The 3D model is a core element of the 3D-PTV method, as it allows us to do the transformation between the coordinates of objects in the 3D world–lab-space–and the 2D coordinates of the camera images–image space–seen by the cameras and vise-versa. In MyPTV there are two such models that have been implemented thus far, each having certain advantages and disadvantages with respect to the other. Other PTV software might use other 3D models. The models implemented are introduced below.

2.1 The Tsai model with non-linear polynomial correction

The 3D model described here is based on the original model that was originally introduced in the seminal work of Maas, Gruen & Papantoniou from ETH Zürich [2]. The basic model uses the physical pin-hole camera model concepts to project 3D points onto an imaging plane; the basic model is invertible, meaning that points on the imaging plane can be transformed back to lines in the 3D world analytically. Notably, this model was the only one used in the first versions of MyPTV (all the 0.x.x versions).

Each camera in this model is represented by its position ($\vec{O}$), orientation vector ($\vec{\theta}$), and the focal length, as described in Fig. 1b. On top of that, to correct for non-linear image aberrations (such as lens imperfection, fisheye, etc), our version of the model uses a non-linear correction term. All in all, the model uses the following equation to transform between the coordinates in the 2D image space coordinates and a direction vector pointing out from the camera imaging center:

$$\vec{r} - \vec{O} = \left(\begin{bmatrix} \eta + x_h \\ \zeta + y_h \\ f \end{bmatrix} + \vec{e}(\eta, \zeta) \right) \cdot \Big[R \Big] \tag{1}$$

Here, $\vec{r}$ and $\vec{O}$ are the position of a particle and the position of the cameras imaging center in 3D space; (η, ζ) are the particle coordinates as seen by the camera in 2D image space; f is a scale factor, equal to the cameras' principal distance divided by the physical size of the camera pixels, $\vec{e}$ is the non-linear correction term, and x_h, y_h are corrections to the cameras' imaging center. The matrix $[R] = [R_1] \cdot [R_2] \cdot [R_3]$ is the rotation matrix calculated with the components of the orientation vector, $\vec{\theta} = [\theta 1, \theta 2, \theta 3]$. The notations are summarized in Table 1. Each camera has its own distinct values of the parameters listed above.

Table 1: Description of mathematical notation. used to describe the 3D model based on the Tsai model

Symbol	Description
$\vec{r}$	Particle position in the lab space coordinates
$\vec{O}$	Position of a camera's imaging center
η, ζ	image space coordinates (pixels) of a particle
x_h, y_h	Correction to the camera's imaging center (in pixels)
f	The camera's principle distance divided by the pixel size
$\vec{e}(\eta, \zeta)$	A nonlinear correction term to compensate for image distortion and multimedia problems.
$[R]$	The rotation matrix which corresponds to the camera orientation vector.

In addition, to the above, the correction term $\vec{e}$ is assumed to be a quadratic polynomial of the image space coordinates:

$$\vec{e}(\eta, \zeta) = [E] \cdot P(\eta, \zeta) = \begin{bmatrix} E_{11} & E_{12} & E_{13} & E_{14} & E_{15} \\ E_{21} & E_{22} & E_{23} & E_{24} & E_{25} \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} \eta \\ \zeta \\ \eta^2 \\ \zeta^2 \\ \eta\,\zeta \end{bmatrix} \tag{2}$$

Here, $[E]$ is a 3×5 matrix that holds the coefficients of the correction term (the last row is filled with zeros because we do not attempt to correct for f).

2.2 The extended Zolof model

The extended Zolof model is used for the transformation between 3D lab-space and 2D image space. Here, the transformation from lab-space to image-space does not involve any physical assumption about how cameras perceive the world—it is purely mathematical. However, a physical assumption is used in the transformation

back from image-space to lab-space. The projection of 3D lab-space coordinates into the 2D image-space is achieved using interpolating functions

$$\begin{bmatrix}\eta\\ \zeta\end{bmatrix} = \begin{bmatrix}P_\eta(\vec{r})\\ P_\zeta(\vec{r})\end{bmatrix} \ . \tag{3}$$

In MyPTV, P_η and P_ζ are polynomial functions of the $\vec{r} = (x, y, z)$ coordinates defined as the product

$$\begin{bmatrix}P_\eta(\vec{r})\\ P_\zeta(\vec{r})\end{bmatrix} = S\,A^\top \tag{4}$$

where S collects the polynomial components, listed in order of ascending degree,

$$\begin{aligned} S\,(x,y,z) = [\,1,\ x,\ y,\ z,\ x^2,\ y^2,\ z^2,\ xy,\ yz,\ xz,\\ xyz,\ xy^2,\ xz^2,\ x^2y,\ yz^2,\ x^2z,\ y^2z,\ x^3,\ y^3\,] \end{aligned} \tag{5}$$

and A is the polynomial coefficients matrix

$$A = \begin{bmatrix}a_0^\eta,\ a_1^\eta,\ \ldots,\ a_{18}^\eta\\ a_0^\zeta,\ a_1^\zeta,\ \ldots,\ a_{18}^\zeta\end{bmatrix} \ . \tag{6}$$

2.2.1 Choosing the order of the forward polynomial

The 19 terms of Eq. (5) are not a complete cubic polynomial in three variables: the depth coordinate z appears at most quadratically, while x and y reach degree three. This asymmetry reflects the geometry of a typical PTV experiment, in which the two in-plane directions are considerably more distorted than the depth direction.

MyPTV allows this choice to be made explicitly. The user selects a triplet of per-axis orders $(o_x,\ o_y,\ o_z)$, each between 1 and 3, and the basis is then the set of monomials

$$x^i y^j z^k \quad \text{such that} \quad i \le o_x,\ \ j \le o_y,\ \ k \le o_z,\ \ \text{and}\ \ i+j+k \le 3\ . \tag{7}$$

The cap on the total degree is separate from the per-axis caps, and both are required. In particular, $(2, 2, 2)$ retains mixed cubic terms such as x^2y while excluding every pure cube, which recovers the 17-term basis used in MyPTV versions 1.3.5 and earlier; $(3, 3, 2)$ recovers the 19 terms of Eq. (5) and is the default; and $(3, 3, 3)$ gives the complete 3D cubic with 20 terms. The number of coefficients per camera is $2n_A$, where n_A is the number of monomials selected by Eq. (7).

It is worth noting that raising a single axis from order 2 to order 3 adds exactly one term to the basis, namely the corresponding pure cube, since the total-degree cap excludes every other possibility. The effect of such a change on the calibration error is therefore usually small. Lowering an axis to order 1, by contrast, removes several terms at once and changes the model appreciably.

2.2.2 The backward transformation

For the transformation from image-space to lab-space we use a unified approach, utilizing a pinhole model in which the direction vector is obtained for each 2D image point via a polynomial. Specifically, intrinsic to this model is the assumption that each 2D camera position $(\eta,\ \zeta)$ is associated with a (straight) line of sight and that all the lines of sight cross at some point in the 3D physical world at the camera position $\vec{O}_i$ of camera i. Therefore, each 3D line of sight for image coordinates from camera i is given as

$$\vec{l}_i(\eta,\ \zeta) = \vec{O}_i + \mu\,\vec{U}_i(\eta,\ \zeta) \tag{8}$$

where μ is a free parameter and $\vec{U}_i$ represents the unit vector associated with each 2D image coordinate of camera i, defined as

$$\vec{U}_i(\eta,\ \zeta) = \mathbf{B}_i \cdot u(\eta,\ \zeta)^\top \ . \tag{9}$$

Here, the matrix $\mathbf{B}_i$ stores the polynomial coefficients of camera i, and the vector u defines the polynomial over the 2D camera coordinates,

$$\mathbf{B}_i = \begin{bmatrix} b_{0x}^{(i)} & b_{1x}^{(i)} & b_{2x}^{(i)} & \ldots & b_{9x}^{(i)}\\ b_{0y}^{(i)} & b_{1y}^{(i)} & b_{2y}^{(i)} & \ldots & b_{9y}^{(i)}\\ b_{0z}^{(i)} & b_{1z}^{(i)} & b_{2z}^{(i)} & \ldots & b_{9z}^{(i)} \end{bmatrix} \tag{10}$$

$$u(\eta,\,\zeta) = \left[1,\,\eta,\,\zeta,\,\eta^2,\,\zeta^2,\,\eta\zeta,\,\eta^2\zeta,\,\eta\zeta^2,\,\eta^3,\,\zeta^3\right] \ . \tag{11}$$

As with the forward transformation, the degree of this polynomial may be chosen by the user. The terms of Eq. (11) are ordered by ascending degree, so a polynomial of order n is obtained simply by retaining the first $n_B = (n+1)(n+2)/2$ terms: the first three terms give a linear model, the first six a quadratic one, and all ten the cubic model used by default. The matrix $\mathbf{B}_i$ then has $3n_B$ coefficients.

2.2.3 The variable origin extension

Equation (8) assumes that every line of sight of a given camera passes through a single point $\vec{O}_i$, that is, that the camera behaves as an ideal pinhole. Real lens systems depart from this assumption, and MyPTV therefore offers an extension in which the origin of the line of sight is itself allowed to vary across the sensor,

$$\vec{l}_i(\eta,\,\zeta) = \vec{O}_i(\eta,\,\zeta) + \mu\,\vec{U}_i(\eta,\,\zeta) \ , \tag{12}$$

where the origin is written as a polynomial in the image coordinates using the same basis u,

$$\vec{O}_i(\eta,\,\zeta) = \mathbf{C}_i \cdot u(\eta,\,\zeta)^\top \ . \tag{13}$$

The order of this polynomial is selected independently of that of $\mathbf{B}_i$, so that the origin may be modelled more stiffly than the direction. This matters in practice: $\mathbf{C}_i$ predicts an unbounded position in lab-space, whereas $\mathbf{B}_i$ predicts a bounded unit vector, which makes the former considerably more prone to overfitting the calibration data.

Two parameterizations of $\vec{O}_i$ are available. In the *free* form, Eq. (13) is used directly and $\mathbf{C}_i$ holds $3n_C$ coefficients. This form carries a redundancy, however: displacing an origin along its own line of sight leaves the line unchanged, so one of the three components is not determined by the calibration data and is fixed only by the smoothness of the polynomial.

The *plane* form removes this redundancy. The origins are constrained to a fixed plane that passes through the mean camera position $\vec{O}_i$, whose normal is directed along the line joining $\vec{O}_i$ to the centroid of the calibration points—that is, a plane facing the measurement volume. Writing $\hat{u}_i$ and $\hat{v}_i$ for two orthonormal axes spanning that plane,

$$\vec{O}_i(\eta,\,\zeta) = \vec{O}_i + \hat{u}_i\,\alpha_i(\eta,\zeta) + \hat{v}_i\,\beta_i(\eta,\zeta) \ , \qquad \begin{bmatrix}\alpha_i \\ \beta_i\end{bmatrix} = \mathbf{C}_i \cdot u(\eta,\,\zeta)^\top \ , \tag{14}$$

so that $\mathbf{C}_i$ holds only $2n_C$ coefficients. Because the plane normal is close to the mean viewing direction, the two in-plane axes span precisely those displacements of the origin that actually alter the line of sight, while the discarded out-of-plane direction is the redundant one identified above. The plane form is therefore both better conditioned and lower in dimension, and it is the recommended choice.

In both forms the image coordinates are centred and scaled before the polynomial is evaluated, which keeps the higher-degree terms of u of order unity across the sensor and prevents the fitted origin from extrapolating wildly outside the region covered by the calibration points.

2.2.4 Parameter count

Each camera is characterized by $2n_A$ coefficients in $\mathbf{A}_i$ for the projection from 3D to 2D, the three coordinates of the camera position $\vec{O}_i$, and $3n_B$ coefficients in $\mathbf{B}_i$ for the backward projection. With the default choices $(o_x, o_y, o_z) = (3, 3, 2)$ and a cubic direction polynomial, this gives $19 \times 2 + 3 + 10 \times 3 = 71$ parameters per camera. When the variable origin is used, a further $2n_C$ coefficients (plane form) or $3n_C$ coefficients (free form) are added, together with the orientation of the plane in the former case; a linear origin polynomial in the plane form, for instance, adds six coefficients.

Since all of these orders are free to be chosen, the appropriate values depend on the number and distribution of the available calibration points. A higher order will always reduce the residual on the points used for the fit, but not necessarily the accuracy elsewhere in the measurement volume. It is therefore advisable to compare candidate settings on points held out of the calibration, rather than on the fitted residual alone.

2.3 Advantage and disadvantage of each model

The extended Zolof model is a later addition to MyPTV, and its main advantage is that it is easier to calibrate than the Tsai model. Specifically, the transformation can be formalized as a linear least square problem such that its globally optimal parameters can be found exactly. In comparison, the Tsai model calibration process must employ a non-convex optimization, making it difficult to converge to the optimal solution. In addition,

the extended Zolof calibration does not require that the user to input an initial guess, which posses one of the main difficulties in the calibration process.

The disadvantage of the extended Zolof model is that it does not comply with a physical principle. Therefore, extrapolation of the model to regions beyond the region of the calibration, or careless use of high degree polynomials (P_η and P_ζ) could lead to errors. The Tsai model on the other hand uses the pin hole camera physical model, and so it is expected to perform better in extrapolations beyond the calibration region.

3 The Workflow: guidelines for a 3D-PTV experiment

MyPTV can be operated using the `workflow.py` script. The script is found under the *example* folder under the home directory of the MyPTV package. The workflow is used through command line given instructions and a file that outlines the parameters to be used in each step.

3.1 Preparation of an experiment folder

To begin post processing of the 3D-PTV results, prepare the following directory structure:

1. Make a new directory; in it, make a new directory called `Calibration` and place inside it the calibration images, and make another set of directories, one for each camera, and place inside it the particle images of each camera.
2. Locate the `example` under the root folder of the MyPTV package. This can be used as a template to the experiment directory made in the previous step.
3. Copy the files `workflow.py` and `params_file.yml` from the `example` folder into your experiment's folder.

3.1.1 The `workflow.py` script

`workflow.py` is a Python script used by the user to run the various steps of the post-processing. The script is used through the terminal or command line. To use MyPTV's various functions, go to the experiment's directory with:

```
cd \path\to\experiment\folder
```

and then, use the following syntax:

```
python workflow.py params_file.yml "command"
```

where `"command"` should be replaced with one of the following options, depending on the particular step of the post processing:

```
"help"
"initial_calibration"
"final_calibration"
"analyze_calibration_error"
"calibration_with_particles"
"calculate_equilization_map"
"calculate_BG_image"
"segmentation"
"matching"
"analyze_disparity"
"tracking"
"smoothing"
"stitching"
"2D_tracking"
"manual_matching"
"fiber_orientations"
"plot_trajectories"
"animate_trajectories"
"run_extention"
```

Each of these commands initiates one of MyPTVs functionalities, and a detailed explanation on them is found in Sections 3.2–3.9. Use `help` to print the available list of commands.

3.1.2 The `params_file.yml` file

The `params_file.yml` outlines all the parameters needed in order to run MyPTV using the workflow script. In every step conducted, go through the lists of parameters and make sure that their values are correct. The definitions per every step are tabulated in the sections below.

Table 2: The `params_file.yml` parameters for the calibration step. All paths to files are relative to the `workflow.py` script.

Parameter	Description
`3D_model`	The name of the 3D model used; can be either `"Tsai"` or `"extendedZolof"`.
`camera_name`	The name of the camera to be calibrated; for example, `cam1`, etc.
`resolution`	Camera resolution; for example: `1280, 1024`.
`target_file`	Path to the file that lists the calibration target's lab points (for example `./Calibration/target_file`)
`calibration_image`	Path of the calibration image (for example `./Calibration/cal1.tif`)

3.2 Calibration guide

A good calibration is key to having success in your PTV experiment! Thus, follow this guide to calibrate your camera using MyPTV.

In the process called camera calibration, our goal is to determine what are the external ($\vec{O}$, $\vec{\theta}$) and internal (f, x_h, y_h, $[E]$) parameters of each of our cameras. This process is done by taking an image, a *calibration image*, of an object that has points with known coordinates marked on it - *calibration target*. Therefore, its important to make sure that for each experiment and each camera we have a calibration image ready, as seen, for example, in Fig. 8a.

Importantly, since our goal in calibration is to find the cameras' positions, once calibration images were taken in an experiment it is crucial that the cameras are not moved. Even minor vibrations can ruin an experiment. Also, the calibration target needs to be in the same medium in which the experiment will be performed to ensure that any refraction is corrected by the error terms (for example, if you are using a tank of water, place the calibration target inside the tank of water).

Once calibration images have been obtains, we use MyPTV to fit the cameras to the 3D-model chosen and estimate the model's parameters. The calibration in MyPTV is performed in two steps: *initial calibration* and *final calibration*. The initial calibration is used to obtain a rough estimation of some of the camera's parameters using a small number of calibration points, as well as to extract the information for all the calibration points on the calibration target. In the final calibration we fine tune the camera's parameters in order to minimize the *calibration error*. The initial and final calibration steps are slightly different for each of the 3D models, so note which model is chosen. The process is explained in detail in the following sub sections.

3.2.1 Preparing a `target_file`

The target file is a text file (tab separated values), in which each row shows the lab space coordinates of a calibration point. An example for the format is shown in Fig. 6b. If there is no target file in the experimental folder, this is the time to generate one. Prepare such a file and save it in the experiment's calibration folder.

3.2.2 Preparing the parameters in the `params_file`

As in any step of MyPTV, we make sure that the `params_file.yaml` in our experiment folder has the correct parameters, namely file names under the calibration tab; see Tab. 2. Each camera is calibrated separately, and so, for the calibration of each camera we need to change the file names in `params_file.yaml` appropriately (alternatively, one could have separate parameter files for each camera).

3.2.3 Initial calibration

To start the initial calibration, we start the *initial calibration GUI*. Using the workflow file in the terminal, use the following command:

```
python workflow.py params_file.yml initial_calibration
```

which should open the a window like the one shown in Fig. 3. The left most panel shows the calibration image; we can zoom in and out by pressing the "+" and "-" keys.

The initial calibration is composed of four steps, and each one has a dedicated panel in the GUI. The steps should, in general, be followed along the order of the steps: $1 \rightarrow 4$.

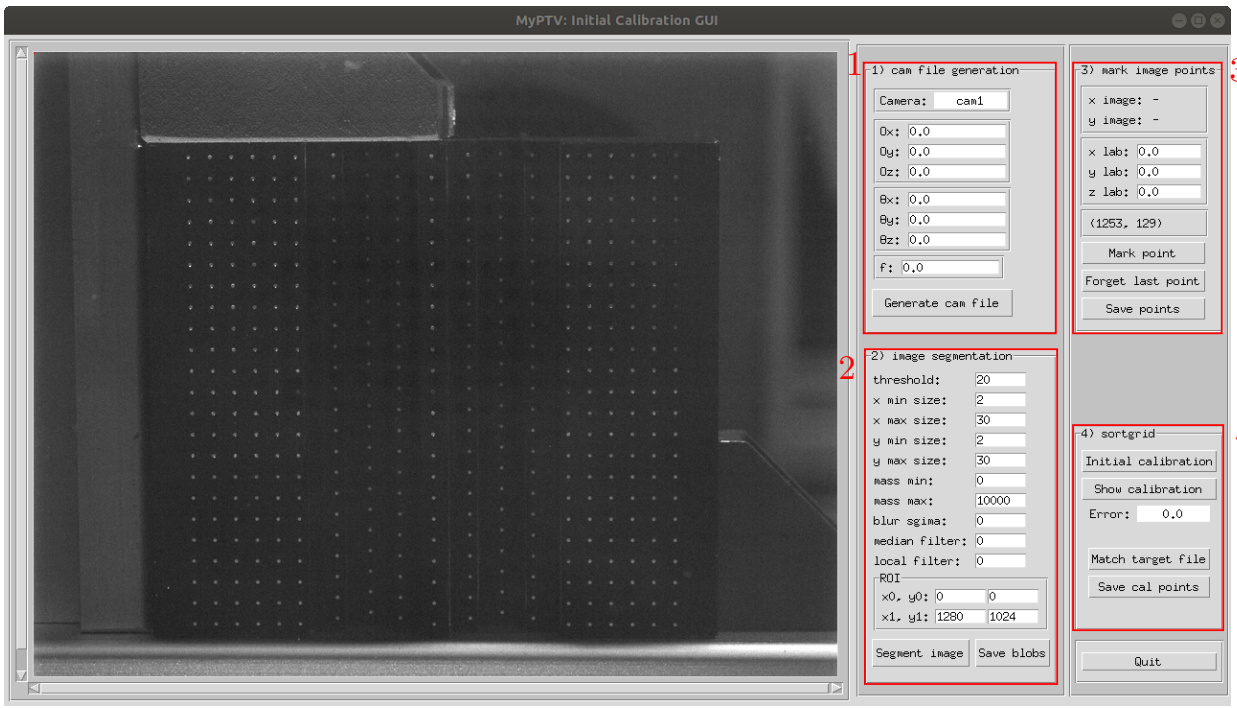


Figure 3: The initial calibration GUI when using the Tsai 3D-model.

Here are the details regarding each of the initial calibration steps:

1. `cam file generation`: Here we produce a text file that stores the camera parameters. If using the *extended Zolof model*, everything is done automatically and the user only needs to click the `Generate cam file` button. If using the *Tsai model*, we first have to give a first guess for the six different external parameters, $\vec{O}$ and $\vec{\theta}$, and the focal length, f. It's important to give a reasonably good initial guess, because the calibration problem often has local minima that make the convergence to solution difficult. To understand the how to choose the values, consult Section 3.2.4. When done, click the `Generate cam file` button.

2. `image segmentation`: In this step we use MyPTV to detect the points on the calibration image and to store their pixel coordinates in a file. Choose and tweak the segmentation parameters until all the calibration points are detected (or at least most of them if the images are not ideal). Detected points are indicated by green squares on the image. Also, a reference for the segmentation parameters can be found in Section 3.3. Once done, save the file using the `save` button.

3. `mark image points`: Here, we start by choosing six points (at least) of the calibration target. The points should not be coplanar and it is best to choose them such that there is some asymmetry to reduce the chance of false convergence. For each point chosen, we mark it by clicking on it on the image and we write its' lab space coordinates in the `x lab`, `y lab`, `z lab` fields, and then click the `Mark point` button. The marked points will be shown as blue crosses on the image. After marking all the chosen points, we save their coordinates using the `Save points` button.

4. `sortgrid`: In this last step, MyPTV tries to calibrate the camera using only the points we marked in step 3, and then to match the calibration points we segmented in step 2 to the points we gave in the target file. Thus, click the `initial calibration` button, make sure that the calibration error is low (aim for about 1.0), and click `show calibration`. If the results are good, click the `Match target file` button, and you should see the segmented points and the associated points from the target file paired by a red line on the image. Make sure that there are no errors in the pairing, and then save the results. If you detect errors or the initial calibration did not reach low enough error, it might be needed to generate an improved initial guess.

This concludes the initial calibration step.

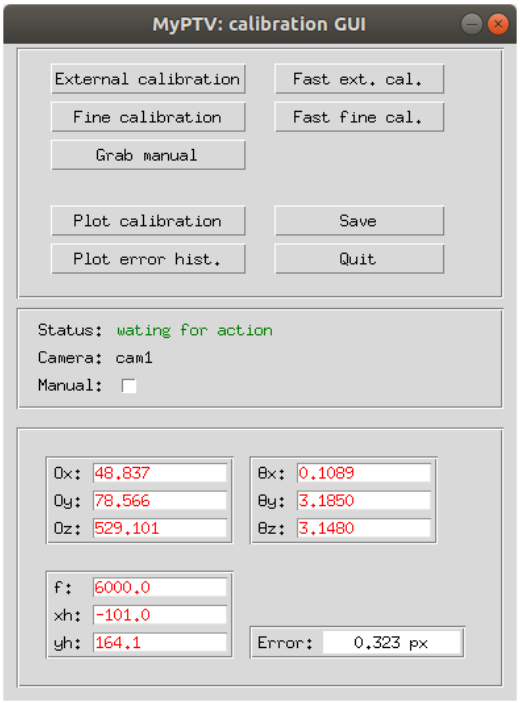


Figure 4: An image of the calibration GUI. The calibration GUI has several functionalities to run the minimization functions, plot the calibration error and the calibration particle's images, as well as change manually the camera parameters. See Tab. 3 for a description of the operation of each button.

Table 3: The various commands available in the calibration GUI (Fig. 4).

Parameter	Description
`External calibration`	Will minimize the calibration error by optimizing the external parameters of the camera using all the calibration points. This might require several runs to reach good results.
`Fast ext. cal.`	Will optimize the external camera parameters using a faster stochastic optimizer. This option is good for rapidly obtaining a reasonable calibration when there are a lot of calibration points.
`Fine calibration`	Will minimize the calibration error by optimizing the nonlinear error term of the camera parameters. This might require several runs to reach good results.
`Fast fine cal.`	Will optimize the nonlinear error terms of the camera parameters using a faster stochastic optimizer. This option is good for rapidly obtaining a reasonable calibration when there are a lot of calibration points.
`Grab manual`	This allows to manually change camera parameters. to do this, first tick the `Manual` checkbox. Then, manually change the values of the camera parameters. Finally, click the button and watch the effect on the calibration error.
`Plot calibration`	Will generate a plot of the calibration points projected on the camera space, and the known calibration points.
`Plot error hist.`	Will generate a histogram of the calibration errors for the various calibration points.
`Save`	Will save the results of the current session in the camera file.
`Quit`	Will quit the GUI.

3.2.4 Tsai model: Understanding camera external parameters

The external parameters of the Tsai model in MyPTV include its position, its orientation, and its focal length, $\vec{O}$, $\vec{\theta}$, and f, respectively. In the calibration process, it is important to understand these parameters to be able to give a good initial guess and judge the results of the calibration procedure. Thus, let us explain how these parameters are defined.

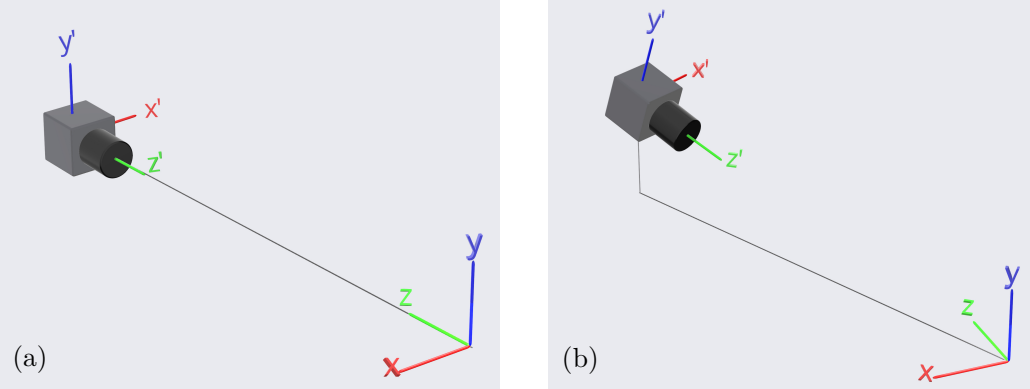


Figure 5: Two examples for camera setups. The (x, y, z) system represents the lab space, and the (x', y', z') represents the camera's system of reference. In (a) the camera's position is roughly $\vec{O} = (0, 0, 1)$, and an appropriate rotation vector could be $\vec{\theta} = (0, \pi, 0)$. In (b) the camera's position might be $\vec{O} = (1, 1, 1)$, and the orientation vector might approximately be $\vec{\theta} = (-\frac{1}{4}\pi, -\frac{3}{4}\pi, 0)$.

Each camera is associated with an image space reference frame, shown in Fig. 5 as the x', y', and z' axes. The z' axis is normal to the imaging plane (the camera's sensor), while x', and y' are tangent to it; the system is right-handed.

The position parameter, $\vec{O}$, is defined as a vector pointing from the origin of the lab space coordinate system to the origin of the camera's reference frame. It is measured in lab-space units (e.g. millimeters). Therefore, one could estimate $\vec{O}$ directly by using a ruler or a measuring tape to measure where the camera is in the lab-space coordinates.

The orientation parameter, $\vec{\theta}$, is a vector of three angles that describe the rotation of the camera's reference frame with respect to the lab space frame, namely the camera's Euler angles. The first angle describes the rotation around the x' axis, the second describes rotation around y', and the third angle describes rotation around z'. Rules of thumb in trying to guess $\vec{\theta}$ are: 1) ask yourself "how should one rotate the lab's frame of reference so that it aligns with the camera's frame of reference?"; 2) we first rotate around the x' axis, then around y' and then around z'. Note: in many cases, the origin of the images is on the top left corner, so half a turn rotation around the z' axis is often needed. See Fig. 5 for an example.

The focal length parameter,f, is essentially an extension factor. In MyPTV f is measured in units of pixels. The focal length is normally more or less equal to the camera's lens focal length. For example, if one uses a camera with a 60mm lens in their experiment, and the size of one camera pixel is 10μm=0.01mm, then f would roughly be equal to 6000 (pixels). Potentially, one could also estimate f in an experiment: 1) place the camera in a known position facing a calibration target (as e.g. in Fig.4a) and take a picture 2) prepare a calibration points file for a few of the points on the calibration target; 3) preparing a camera file using the known position and orientation 4) insert some initial value for f; 5) using the `workflow.py` file to compare the distribution of the points using this value of f; 6) repeat steps 4 and 5 until a fair agreement is found.

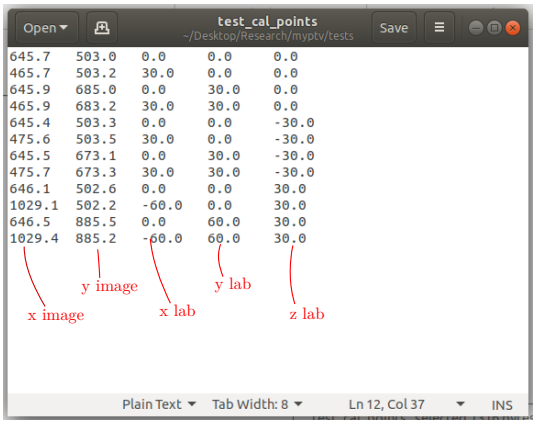
3.2.5 Final calibration

In the final calibration we use all the calibration points to find the parameters of the camera. We do this by minimizing the calibration error, defined as the root mean square of the distances between the image-space calibration points positions and the projection of the lab-space points on the image plane. The procedure is performed using the final calibration gui, which is initiated by using the workflow script with the command

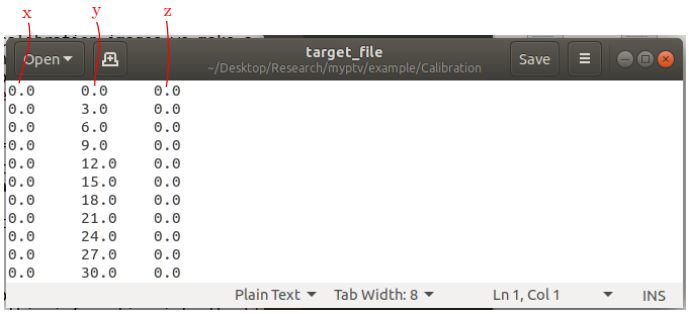
```
python workflow.py params_file.yml final_calibration
```

The final calibration gui is different for each 3D-model. For the *extended Zolof*, the minimization is done by solving an exact least squares problem, and for a given set of calibration points there is only one solution which is found by the solver. Therefore, to complete the calibration simply click on the `calibrate` button, and once the process is finished the calibration was found. Click `save` to save the results into the camera file. You can inspect the calibration using the histogram and the calibration target plots.

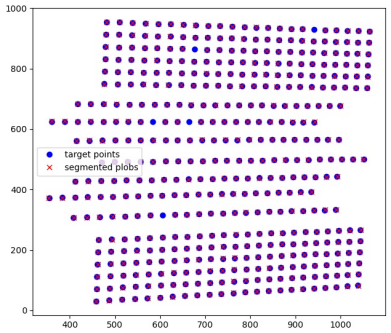
For the *Tsai model*, the minimization is solved numerically in iterations using the *final calibration gui*. The gui for the Tsai model is shown in Fig. 4. The description of each of the possible functionalities are given in Table 3. In principal, the lower the calibration error, the results of the measurements will be more reliable, and



(a)



(b)



(c)

Figure 6: (a) Format of the target file, which lists the lab-space coordinates of the calibration target as tab-separated values. The order at which points are given is not relevant. (b) An example of a text file holding the calibration point data. (c) The results of matching a target file points (blue circles) and the segmented blobs (red crosses). Here, we know that the match is correct because each cross overlays a circle. In location where circles are not overlaid by a cross, the segmentation did not recognize a calibration target; having such missing points may slightly reduce the quality of the calibration but it is not critical for the target file matching.

it will become possible to track a higher number of particles in each frame. Once a sufficiently low error value is achieved, the calibration process is complete.

Note: A "good" calibration will have a low error value - how much exactly is difficult to say and it strongly depends on the imaging setup used and in particular the imaging resolution. Typically, we would want to get down to approximately 1 pixel or lower.

3.2.6 Calibration error analysis, validation and uncertainty

Once all the cameras have been calibrated, it is important to verify the calibration quality, as this gives us information on the uncertainty of the results. We analyze the calibration error by stereo matching the calibration points, which is done through the workflow command "`analyze\_calibration\_error`".

The `analyze calibration error` command calculates statistics of the deviation between the ground truth of the calibration points and the estimate that is obtained using stereo matching. The command outputs two types of information:

1. it prints the root mean square of the deviations both in total distance and for each direction;
2. it plots histograms of the deviations' total distance and in each direction.

The results are shown in lab-space coordinates (i.e. in millimeters). Therefore, the values obtained can be taken as the lower bound for the uncertainty in the measurements of the particle positions in the experiment. Figure 7 shows an example for the output of this type of analysis.

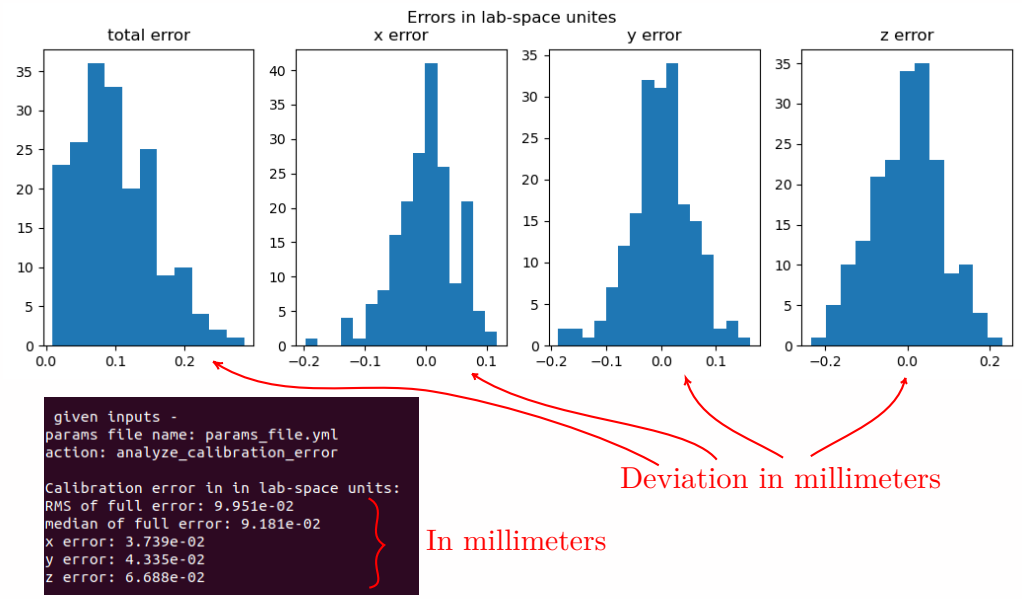
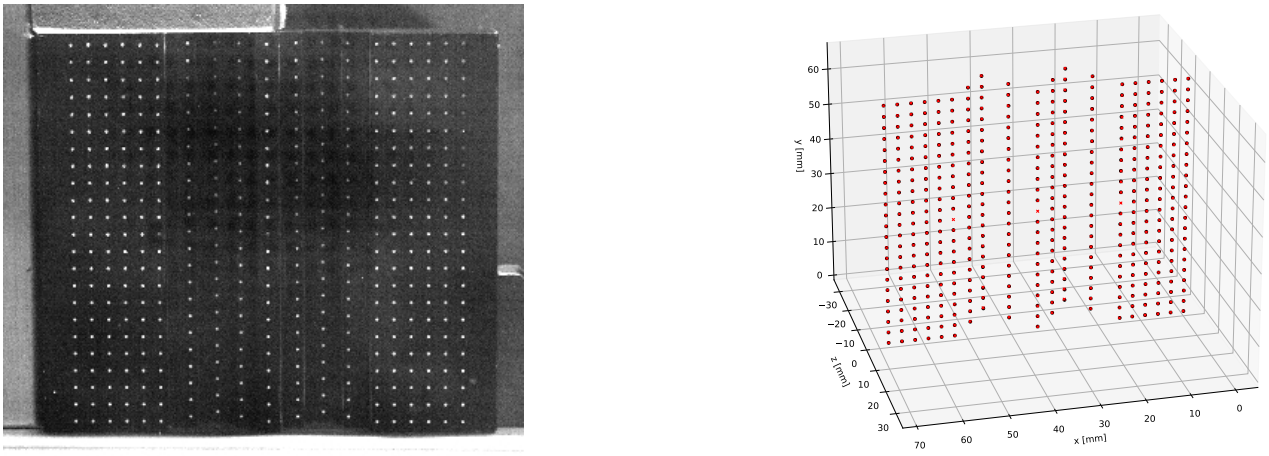


Figure 7: Example results of the calibration error analysis. In this experiment, the lower bound for the uncertainty for position measurements based on the RMS deviation value was approximately 100 μm, with the largest deviations in the z direction.



(a) (b)

Figure 8: (a) An example of a calibration image. The points on the target have known lab space coordinates. Not also that the points are distributed over several plains (3 different z values in this case). (b) An example plot of a calibration error estimation. The red crosses represent the known positions of the calibration points given in the target file, and the black circles mark the positions of the stereo-matched segmented calibration points. In this case, the root mean square of the static calibration error was 84μm.

3.2.7 How to perform calibration with several images (sometimes called multiplane calibration)

Calibrating the PTV cameras requires having calibration points spanning a full 3D volume, with points being positioned on different planes. Traditionally, this is done by constructing the calibration target with multiple planes (Fig. 8a). However, in some PTV systems another solution is taken such that the calibration target is moved to different locations and images of the target are taken there. In OpenPTV, this is called multiplane calibration. The following steps outline the procedure to follow when using such a multiplane calibration in MyPTV:

1. Place the calibration target in its first position; make sure you the location of the target and take an image of it. Then, move the target to another position and take an image of it. Repeat until the whole domain is covered by your calibration points, and save the sequence of images you just took.

2. Prepare a target file for each one of the images in the calibration sequence (see Seq. 3.2.1). Note that the

target file for each image needs to hold only the information for the points that are seen in its particular image.

3. Perform an initial calibration for each of the calibration images, each with respect to its own target file. When you do so, make sure that the you are using a different "camera name" in the params_file (for example, "cam1_1", "cam1_2", "cam1_3", ..., cam1_i).
4. After finishing the previous step, for each calibration image you should have a file called "XXXXX_i_cal_points" where "XXXXX_i" stands for the camera names you put in the params file.
5. Make a new empty file called as the base name of the previous files (the "XXXXX" part of the "XXXXX_i"); in the params_file, change the camera name to the same value ("XXXXX"). Inside the new file just generated, copy and paste the data from all the "XXXXX_i_cal_points" files.
6. Make a copy of one of the camera files, and change its name to "XXXXX". This will make sure you have a camera file from which we start the final calibration.
7. Run the final calibration as done in the regular calibration procedure.

3.2.8 Calibration from checkerboard images

The calibration described so far obtains its calibration points by hand: the user clicks a number of points of the target in the initial calibration GUI, and the remaining points are then matched to the `target_file` automatically. This subsection describes an alternative way of obtaining the same calibration points, from images of a checkerboard target, in which the points are found without any clicking.

It is worth being clear about what this replaces. The checkerboard procedure produces an ordinary calibration points file, of exactly the form described in Sec. 3.2.1, holding the pixel coordinates of each point and its lab coordinates. It does not replace the fitting of a 3D model, and it makes no assumption about which model is used: once the file exists, the model is fitted from it with the `final_calibration` action, exactly as it would be from a hand picked file, and equally well with the Tsai model or with the extended Zolof model. The checkerboard procedure therefore replaces the point picking, and nothing else.

The experiment. The procedure expects the arrangement that is usual in 3D-PTV. A checkerboard is mounted on a translation stage inside the measurement volume, oriented so that its two sides lie along known directions of the lab frame. It is then photographed by all the cameras simultaneously at a number of known positions of the stage, so that its corners between them span the depth of the volume. This is the same multiplane idea as in Sec. 3.2.7, and it has two useful consequences: because the cameras record the board at the same instant they share one lab frame by construction, and because the stage positions are known the depth coordinate of every corner is known to the accuracy of the stage, rather than having to be fitted.

The board is described to the program by the number of its **internal** corners, by the physical spacing of those corners, and by its pose. Only the internal corners are used, since a corner on the outer boundary of the pattern is not surrounded by the board on all sides and cannot be located reliably; a board of 10×7 squares therefore has 9×6 internal corners, and `board_size` would be `9, 6`.

How the corners are found. An internal corner of a checkerboard is a saddle point of the image intensity, and candidates are found as the maxima of $-\det(\mathsf{H})$, where H is the Hessian of the smoothed image. Each candidate is then refined to sub-pixel accuracy using the fact that at any pixel p lying on one of the two edges through the corner c, the intensity gradient is perpendicular to $p - c$; minimizing the squared residuals of $g(p) \cdot (p - c) = 0$ over a window around the corner gives a small linear system for c. The two edge directions of each corner are recovered from a histogram of the gradient orientations around it, and are used both to reject candidates whose surroundings do not look like a checkerboard and, afterwards, as the local lattice directions.

The refined corners are then assembled into an ordered grid by growing it outwards from a seed corner, in the spirit of Ref. [3]: at each step the image position of a lattice site next to an already filled one is predicted from its filled neighbours, and the nearest unused corner is assigned to it if it falls close enough to the prediction. Growing the grid locally, rather than fitting a global model to it, means that the procedure follows perspective and lens distortion without having to know anything about either, and that it stops of its own accord at the edge of the board. Tested on synthetic images, the corners are recovered to within about a twentieth of a pixel of where the camera model projects them.

The orientation of the board. One point deserves attention, because getting it wrong produces a calibration that is badly wrong without any obvious symptom. The grid that comes out of the growth is indexed arbitrarily:

Table 4: The `params_file.yml` parameters for the **segmentation** step. All paths to files are relative to the `workflow.py` script.

Parameter	Description
`image_start`	The number of the image from which the loof begins. If this is `None` (default) the first image is set to 0.
`Number_of_images`	Number of images over which to do the segmentation. If it is 1, segmentation is done only on the image with the name given in `single_image_name`. If this is an integer, segmentation is performed on the first N images found in the directory `images_folder`. If this is `None`, segmentation is performed over all the images in the directory `images_folder`
`images_folder`	path to the folder containing the images
`single_image_name`	If `Number_of_images` is 1, the segmentation will be done on the image whose name is given here. If `Number_of_images` is any other value, this parameter is not used.
`image_extension`	extension of the images; for example, .tif
`raw_format`	Set this to True if your images are in RAW format (e.g. .dng), and False otherwise (usually this should be set to False)
`shape`	Can be either 'particles' or 'fibers'. The 'particles' option is used for isotropic particles to follow traditional PTV, and the 'fibers' option is used for analyzing the orientations in 3D of anisotropic particles (elongated fibers or rods), see Sec. **??**.
`pca_limit`	This parameter applies only when `shape` is set to 'fibers'. It specifies the maximum allowable ratio of eigenvalues from the PCA decomposition of fibers in image space. For values greater than 1, only fibers with an aspect ratio below this threshold will be segmented, and particles in the same frame will be ignored.
`plot_result`	if `False` will not plot the results; if `True` the results will be plotted but only if number of images is 1
`mask`	if this is 1.0, no mask is used; if this is set to a path to a file, the file will be used as a mask for the segmentation, where the file should be a black and white image where white marks the region of interest
`ROI`	region of interest; specify by indicating the xmin, xmax, ymin, ymax coordinates
`threshold`	the brightness value for which a point is considered a particle after a local mean subtraction is performed
`remove_background`	If string, then it is the path to a pre-calculated background image (see sec. 3.3.2). If False, background subtraction is not used.
`equilization_map`	If string, then it is the path to a pre-calculated equalization map (see sec. 3.3.2). If None, equalization is not applied.
`DoG_sigmas`	Either a list of two numbers $[\sigma_1, \sigma_2]$ that represent the widths of two blurring operations, or None, in which case the filter is not applied.
`median`	the intger window size of a median noise removal filter; set to `None` in order to not apply the filter
`blur_sigma`	the float standard deviation of a Gaussian blur filter; set to `None` in order to not apply the filter
`local_filter`	the scale of Gaussian filter used to normalized brightness intensity (see 3.3.4); set to `None` in order to not apply the filter
`min_xsize` or `min_ysize`	minimum particle size (pixels) in x or y direction
`min_mass`	minimum particle mass (mass is the sum of pixel grey values)
`max_xsize` or `max_ysize`	maximum particle size (pixels) in x or y direction
`max_mass`	maximum particle area (mass is the sum of pixel grey values)
`plot_result`	if `True` the segmented particles will be plotted over the processed image (after applying blur, local mean subtraction and masking); if `False` will not plot the results
`method`	The name of the segmenting method; can be either 'labeling' or 'dilation'
`particle_size`	in the 'dilation' method, this gives the diameter of blobs in the image (an integer number of pixels); this parameter is not used when method is set to 'labeling'
`save_name`	if `None` the results will not be saved in a file; if `path/to/file` will save the results in the given file name

which corner ended up being called $(0, 0)$, and which side of the board was called the first axis, depend on where the growth happened to start. A grid can therefore be laid over the physical board in eight different ways. If

two cameras resolve this differently, they assign different lab coordinates to the same physical corner, and the calibration silently fails.

The ambiguity is resolved by the `origin_hint` parameter, which is the approximate pixel position, in that camera, of the corner that the user has chosen to be the origin of the board. It need only be accurate to within a fraction of the corner spacing. For a board whose two sides have different numbers of corners this fixes the orientation completely; for a square board a second hint, `iaxis_hint`, pointing along the first axis, is also needed. **The same physical corner must be hinted at in every camera.** It is worth looking at the figure that the action plots, in which the chosen origin is circled and the two axes labelled, once per camera, before going on to fit the models.

Running it. With the parameters of Tab. 5 set, the points are found with

```
python workflow.py params_file.yml checkerboard_calibration
```

once per camera, after which the models are fitted with `initial_calibration` and `final_calibration` as usual. A set of synthetic checkerboard images can be generated from existing camera files with the script `synthetic_data_generation/checkerboard/checkerboard_image_generation.py`, which is a convenient way of becoming familiar with the parameters before relying on them.

3.2.9 Calibration from a freely moved checkerboard

The procedure of Sec. 3.2.8 needs the board on a translation stage, so that its position is read off the stage and the lab coordinates of every corner are known before any image is looked at. This subsection describes the other common way of using a checkerboard: the board is simply held in the measurement volume and turned about between exposures. Nothing is then known about where it was, and its pose in each frame has to be recovered from the images, together with the cameras themselves.

What the procedure needs in exchange is two things. The cameras must be **triggered together**, so that a given frame shows one pose of the board seen from several directions at once; this is what ties the cameras into a single lab frame. And the board must genuinely be **turned about** between exposures, not merely slid around, because the internal parameters of each camera are recovered from the way the perspective changes with orientation, and images differing only by a translation carry the same information over and over.

How it works. The corners are found and ordered exactly as before (Sec. 3.2.8). Then, for each camera separately, the homography of the board plane onto the image is fitted for every frame, and the internal parameters follow from the orthonormality constraints that Zhang [4] derived; with the internal parameters known, each homography gives the pose of the board relative to that camera. Those per camera answers are noisy, however, and they are not consistent with one another, since each camera reconstructs the board on its own. The last step therefore adjusts everything together, minimizing the distance in pixels between the corners found and the corners the model predicts, over

- the position, orientation and internal parameters of each camera, and
- the pose of the board at each frame.

The essential point is that the board pose at a given frame is *one* set of six numbers, shared by every camera that saw it. A corner is then no longer reconstructed separately by each camera but triangulated by all of them at once, and cameras placed well apart determine its depth far better than any one of them could alone.

Which way round is the board? The lattice that comes out of the corner ordering is indexed arbitrarily, and unlike the stage case there is no fixed pixel position to hint at, because the board moves. The ambiguity is resolved from the geometry instead. The shape of the board rules out interchanging its two sides unless it is square. Of the four remaining relabellings, two describe the board seen from behind, and they are recognised because the pose that comes out of them has the board's face turned away from the camera. The last two differ by a half turn about the board normal, and they are separated by requiring the transformation between any two cameras to come out the same at every frame, which it must, since the cameras do not move. That requirement is also a strong check on everything upstream of it, and the action reports how many frames agree.

The lab frame is a convention. This deserves emphasis, because it is easy to mistake for a fault. There is nothing in a set of images of a board being waved about that says where the origin of the lab frame lies or which way its axes point. The *scale* is fixed, by the physical size of the squares, so distances, angles and shapes all come out right. The frame itself does not: the procedure simply declares one camera to be the origin. A user who measures an absolute coordinate against a ruler and finds it a few millimetres out has not found an error;

Table 5: The `params_file.yml` parameters for the `checkerboard_calibration` step. All paths to files are relative to the `workflow.py` script.

Parameter	Description
`camera_name`	The name of the camera whose images are being processed; for example `cam1`.
`images`	The images of the board for this camera, either as a list or as a pattern such as `./Calibration/cam1_board_*.tif`. A pattern is expanded in sorted order, so it is worth numbering the files with a fixed number of digits.
`board_size`	The number of **internal** corners along the two sides of the board; for example `9, 6` for a board of 10×7 squares.
`square_size`	The physical spacing of the corners, in the length units used for the lab coordinates. Two values may be given if the spacing differs along the two sides.
`board_origin`	The lab coordinates of the corner $(0, 0)$ of the board, for the board at zero translation; for example `7.0, 17.5, 0.0`.
`board_i_axis`	The lab space direction of the first side of the board; for example `1, 0, 0`.
`board_j_axis`	The lab space direction of the second side of the board; for example `0, 1, 0`.
`translations`	One value per image, giving the position of the translation stage when that image was taken, measured along the normal of the board; for example `-8.0, -4.0, 0.0, 4.0, 8.0`.
`origin_hint`	The approximate pixel position, in this camera, of the corner chosen as the origin of the board; for example `339, 703`. This must be the same physical corner in every camera.
`iaxis_hint`	The approximate pixel position of a corner along the first axis of the board. Needed only when the board is square, where the origin alone leaves two possibilities; `None` otherwise.
`sigma`	The smoothing scale of the corner detector, in pixels. It should be of the order of the blur of the edges of the board; 2.0 is a reasonable starting value.
`min_distance`	The minimum separation of two corners, in pixels. It must be smaller than the spacing of the corners in the image, and also sets the size of the windows in which the corners are refined.
`min_score`	Candidates whose surroundings match an ideal checkerboard corner less well than this are discarded. The default of 0.7 separates true internal corners, which score above about 0.85, from the spurious candidates found on the outer boundary of the pattern, which stay below about 0.55.
`plot_result`	If `True`, plots the corners found in the first image with the chosen origin circled, which is the check that the orientation of the board came out as intended.
`save_name`	The name of the calibration points file to write. If it has no directory part it is written into the calibration folder.

the frame is elsewhere. If the coordinates must line up with something physical, a tank wall or a traverse, then either relate the recovered frame to the apparatus afterwards using a few measured points, or place the board deliberately for one exposure and adopt that as the reference.

Running it. With the parameters of Tab. 6 set,

```
python workflow.py params_file.yml moving_board_calibration
```

processes all the cameras at once, unlike the other calibration actions, since they have to be solved together. It writes a calibration points file per camera and, when the Tsai model is asked for, a camera file as well: the parameters that come out of the adjustment are already a good fit, so the initial calibration GUI of Sec. 3.2.3 is not needed at all. Check the result with `analyze_calibration_error`, and refine further with `final_calibration` if wished.

A worked example is provided, of fourteen frames from four cameras of a real board being turned about, in `example/checkerboard_moving_example`. Its `README` lists the numbers it should reproduce, including the two that are independent of the choice of lab frame: the two pairs of cameras come out 246.1 mm and 245.1 mm apart, recovered independently of each other, and triangulating the corners back through `img_system` returns the spacing of the board as 14.97 mm against its true 15 mm. Synthetic images for either procedure can also be generated from existing camera files with `synthetic_data_generation/checkerboard/checkerboard_image_generation.py`, choosing between them with its `mode` setting.

What to look at afterwards. The action prints several things worth reading rather than skipping.

- The *reprojection error*. It should approach the per view homography residual, which is also printed and which is the error of the corners themselves plus whatever lens distortion the model cannot absorb. On a real four camera set of an 8×11 target these were 0.14 px and 0.137 px respectively.
- The *worst views*. One view far worse than the others almost always means its corners were labelled the wrong way round. Drop that frame and solve again.
- The *distances between the cameras*. No choice of lab frame can change these, so they are a fair check. If two cameras are on a common mount whose separation is known, comparing it with the recovered value is a genuinely independent test. On the set mentioned above, two pairs of cameras came out 245.4 mm and 244.8 mm apart, recovered independently of each other.
- The *region the calibration points fill*. Outside it the calibration is extrapolation, and it should be checked against the intended measurement volume.

A further check, once the camera files are written, is to triangulate the calibration points back with `img_system` and see whether the spacing of the board comes out at its physical value. This does not test the scale, which was set by declaring the square size, but it does test everything downstream of the adjustment. On the set above it returned 15.0009 ± 0.0247 mm for a 15 mm board.

3.3 Segmentation

Segmentation is essentially the image analysis step of the PTV method, in which we extract the particles' image-space coordinates from the experiment images and save the coordinates in dedicated files.

3.3.1 How to segment in MyPTV

The basics of the segmentation step are done via the `workflow.py` script with the `segmentation` command. To run the process follow these steps:

1. We perform the segmentation first on a single image by specifying `Number_of_images: 1`, and giving an image name with `single_image_name`. We then tune the segmentation parameters and the image processing filters used (e.g. `ROI`, `threshold`, `blur_sigma`, `method` etc (details below); the full list is given in Table 4) to obtain optimal results; optionally repeat over several more images to ensure the results are satisfactory. To view the segmented particles over the image, use `plot_result: True`, and make sure the results are not saved by using `save_name: None`.
2. Once the parameters are determined, we set `Number_of_images: None`, and run the segmentation workflow again. This will then loop over all the images in the given folder (under `images_folder`), and save the results in a text file by setting `save_name: /file/name/to/use`

Table 6: The `params_file.yml` parameters for the `moving_board_calibration` step.

Parameter	Description
`camera_names`	All the cameras, which are calibrated together; for example `cam1, cam2, cam3, cam4`. At least two are needed, since a single camera cannot fix a lab frame.
`images`	Where the images are, with `{camera}` standing for the camera's name; for example `./Calibration/{camera}/*.tif`. The frames of different cameras are matched by their position in the sorted list, so each camera must have the same number of images, in the same order, and the exposures must correspond.
`resolution`	Camera resolution, needed only for the Tsai model, which holds the principal point as an offset from the centre of the image.
`3D_model`	`Tsai`, in which case camera files are written as well as calibration points, or `extendedZolof`, in which case only the points are written and the model is fitted from them with `final_calibration`.
`board_size`	The number of **internal** corners along the two sides of the board; `8, 11` for a board of 9×12 squares.
`square_size`	The physical spacing of the corners. This is what fixes the scale of the whole result, so it should be right.
`every_nth_frame`	Use only every nth image. A long sequence of a board being waved about contains far more views than a calibration needs, and detection is the slowest part.
`max_frames`	How many frames to use in the adjustment, taken well spread through those that were usable. Ten to thirty views of the board in different orientations is ample; more only costs time.
`reference_camera`	The camera whose position and orientation define the lab frame, and which is therefore held fixed.
`n_distortion`	How many distortion coefficients to fit: 0, 2 (two radial terms) or 4 (two radial and two tangential).
`sigma`, `min_distance`, `min_score`	The corner detector, as in Tab. 5.
`max_iterations`	The limit on iterations of the adjustment.
`save_folder`	Where to write the calibration points files.

3.3.2 Image pre-processing

Image analysis is a key step in PTV and therefore, there are many algorithms used to improve the results of this step. Some of these methods require preparations that need to be made before the segmentation, and these preparations can be called pre-processing. There are two possible pre-processing steps currently available in MyPTV:

1. ***Background images***: In some cases where the imaging conditions are not perfect, background objects will appear in the images and obstruct the segmentation. To negate these effects we calculate a "static background image" (defined as the mean of the images) and during the segmentation this background image is subtracted from each image. This is done using the workflow command `calculate_BG_image`, which is used to calculate the mean image and save it on the disk for later use during segmentation.

   The background image is defined here as the arithmetic mean of images from the image folder:

   $$\mathrm{BG} = \frac{1}{N_{im}} \Sigma \mathrm{Im}_i \ ; \tag{15}$$

   After that, the images are processed by taking the difference with the background

   $$\mathrm{Im}' = |\mathrm{BG} - \mathrm{Im}| \tag{16}$$

2. ***Intensity equalization***: In some cases where illumination is not uniform, some particles will have different intensities in different regions of the images and thus will make segmentation more challenging. To negate this, we can create an equalization map", which attempts to equalize the brightness intensity of the particles at different locations in the images. We prepare this equalization map with the workflow command `calculate_equilization_map`. The map is then saved as a tif image on the disk and is used during the segmentation step to improve the segmentation results.

   The equalization map is defined here be calculating the pixel-wise maximum intensity across all the images, and then applying a gaussian blur with a given standard deviation

   $$\mathrm{EQmap} = \frac{\mathrm{GaussianBlur}(Max(\mathrm{Im}_i), \sigma)}{Max\left[\mathrm{GaussianBlur}(Max(\mathrm{Im}_i), \sigma)\right]} + \epsilon \tag{17}$$

   After that, the images are processed by dividing each image with the equalization map

   $$\mathrm{Im}' = \mathrm{Im}/\mathrm{EQmap} \tag{18}$$

An example for the result of these two steps is shown in Fig. 9. To actually apply these methods during segmentation, the background image and the equalization map are given as inputs during the segmentation process as detailed below.

3.3.3 Two methods of segmentation

In the segmentation step, the program extracts the location of particles in the image. Segmentation in MyPTV is performed in one of two methods: ***labeling***, or ***dilation***, and includes several optional image processing steps.

- **Labeling:** In this method, we first choose all pixels in an image whose gray value is higher than a given threshold. Then, such pixels that are touching each other are considered to be "blobs", so we group them together. For each blob, we calculate a *center of mass*, being the gray value weighted average of the blob, a *bounding box* size, and a *mass*, being the sum of gray values of the blobs' pixels.
- **Dilation:** Searching for objects of a certain size. In this method, we are looking for pixels that their gray value is higher than that of all their neighbors within a box of a given size (`particle_size`), and that their gray value is higher than a given threshold. After that, all the neighboring pixels inside the particle size box are considered to be a blob. We then calculate the center of mass of the blob to achieve a better estimation of its position, thus redefining the blob. This process is iterated a maximum of 3 times until convergence is obtained. Finally, we calculate the same blob's *center of mass*, *bounding box* size, *mass*.

While the dilation method can take more computational time, it can sometimes be used to solve segmentation of large objects where overlap is an issue (see Fig. 10).

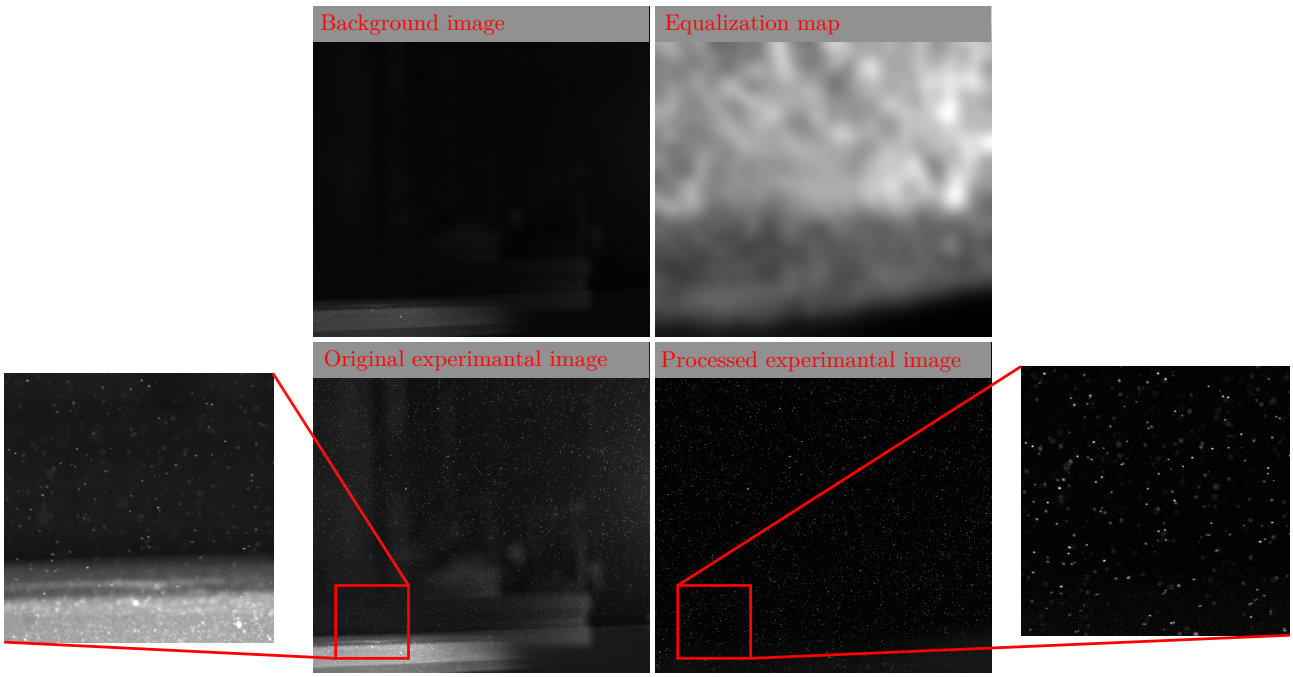


Figure 9: Examples of background image, equalization map, original experimental image and the same image after background subtraction and intensity equalization.

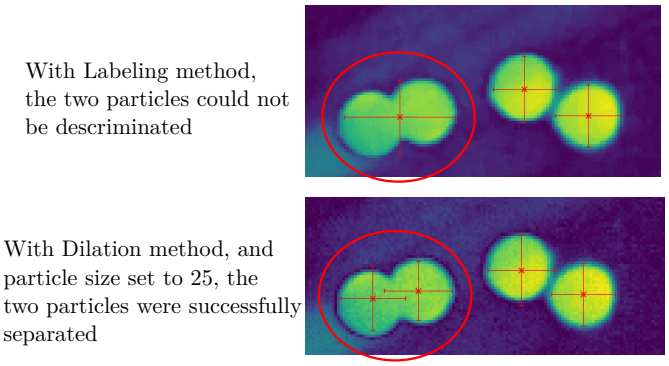


Figure 10: An example showing that segmentation using the dilation method can be very useful when segmenting large objects as it can resolve overlap issues.

3.3.4 Segmentation filters

To improve the results of particle segmentation in non-ideal images, MyPTV can be set to apply image processing before the blob extraction. The filters that can be used are:

1. **Gaussian blur** - blurring the image with a given scale, used to remove salt and pepper kind of noise.
2. **Median filter** - subtraction of the local median in over a pixels local neighborhood.
3. **Local filter** - used to filter out low intensity regions relative to their neighborhood; each pixel brightness is normalized with the local mean and variance at a given scale (blur standard deviation), and the image is then conditioned to pass only local high brightness pixels.
4. **Mask** - masking out of regions not needed for the analysis.
5. **DoG filter** - "Difference of Gaussian", the image is blurred with a gaussian filter once with one standard deviation and another time with another standard deviation and the two blurred images are then subtracted from one another.

These filters are often necessary to properly segment the particles in an image depending on the experimental conditions, such as uneven illumination or noisy images. Examples of the results of using several combinations of the filters on a calibration image is shown in Fig. 11.

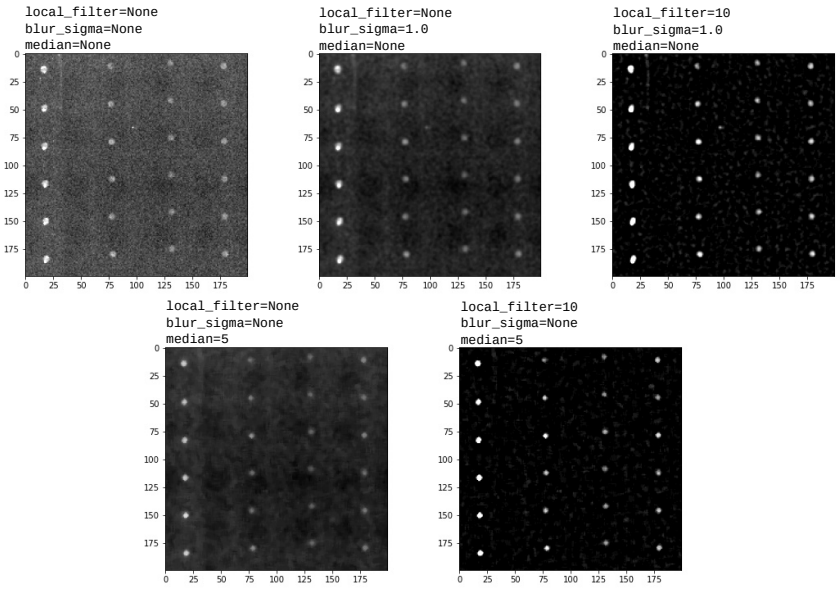


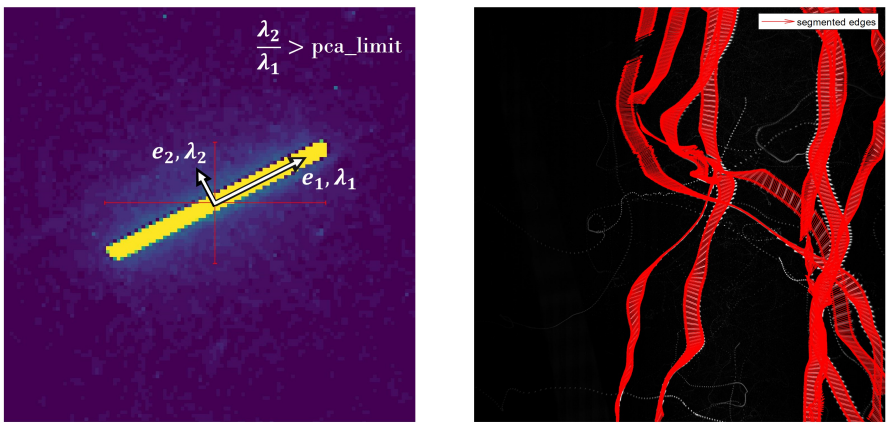
Figure 11: The effect oft various image filters on the segmentation.

3.3.5 Fiber segmentation

This module segments both positions and orientations of elongated particles in the image, here called *fibers* or *rods*. The segmentation principle for center recognition remains the same as for particles from section 3.3, while the 2D orientation is found with the principle of Principal Component Analysis (PCA). This method consists on selecting the pixel clusters making up a fiber blob and calculate the 2×2 covariance matrix $\mathbf{C}$ using the matrix of pixel cluster coordinates centered at $\widetilde{\mathbf{X}}_i = [x_i - \langle x \rangle, y_i - \langle y \rangle]$

$$\mathbf{C} = \widetilde{\mathbf{X}}^T \widetilde{\mathbf{X}},$$

where $[\langle x \rangle, \langle y \rangle]$ is the image fiber centroid. By performing an eigenvalue analysis of $\mathbf{C}$, i.e. $\mathbf{C}\mathbf{e}_i = \lambda_i \mathbf{e}_i$ with $i = \{1, 2\}$, the eigenvector $\mathbf{e}_1$ corresponding to the largest eigenvalue λ_1 denotes the orientation vector of the fiber in the image (see Fig. 12(a)).



(a) (b)

Figure 12: (a) An example of the segmentation of fibers in image space with the PCA analysis. (b) Consecutive superimposed images of heavy fibers settling in turbulence, with `pca_limit` = 1.3. As seen in the images, only fibers are segmented while particles in the background are left out.

An additional feature of the PCA analysis is that the ratio between the larger and smaller eigenvalue λ_2/λ_1 gives an estimate of the fiber aspect ratio in image space. This parameter (`pca_limit`) is used as a threshold to differentiate particles and fibers in the same image (see Fig. 12(b)).

The latest version of this code uses the functions `regionprops` and `labelski` from the `skimage`[1] library available

[1] https://scikit-image.org/

for Python.

3.4 Matching

In the matching step, particles that were segmented from the various images are used to stereo-locate (triangulate) the 3D positions of particles in the lab-space coordinates.

One of the big challenges in the PTV method is to determine which particle in each image correspond to the same particle in other images. As the number of combinations can be very large for particle dense images this can be a time consuming computation. In MyPTV, we approach this issue using a new algorithm called "particle marching". In brief, the algorithm begins by assuming we have an initial guess for particle positions in an image, and we then check whether these initial guesses correspond to physical particles using the co-linearity condition. If an initial guess turns out to be correct, we store its location and if not, we throw it out. Then, in the next frame we use the particles found in the previous frame as new initial guess. For the first iteration we divide the measurement volume into smaller sub-volumes (called voxels, with a given size), and search for particles inside each voxel using the ray-traversal algorithm described in [5].

Getting the voxel size right was seen to yield good results. If it is given too large this step could take up significant computational time. To optimize its value, run first only on several images and only after that iterate over all frames by setting `N_frames: None` and save the results.

Note: to easily perform manual stereo matching of several points in a given image, use the manual matching GUI. See Section 3.11 and Fig. 14.

Table 7: The `params_file.yml` parameters for the **matching** step. All paths to files are relative to the `workflow.py` script.

Parameter	Description
`blob_files`	Names of files that hold the segmented particles for each camera, separated by commas; for example: `blobs_cam1, blobs_cam2, blobs_cam3`
`frame_start`	If `None` will start matching from the first available frame. If an integer it will start the matching from this given number.
`N_frames`	If `None` will match particles in all available frames; if an integer will match only particles in the first `N` frames.
`march_forwards`	If `True` then the particle marching sequence includes a pass forwards in time.
`march_backwards`	If `True` then the particle marching sequence includes a pass backwards in time.
`camera_names`	Names of cameras used separated by commas; for example, `cam1, cam2, cam3`
`cam_resolution`	Camera resolution; for example, `1280, 1024`
`ROI`	Region of interest for the matching in lab space coordinates, and according to the format of `xmin, xmax, ymin, ymax, zmin, zmax`
`voxel_size`	Can be either a float or None. If it is a float, then this is the side length of voxels used in the *Ray Traversal* algorithm; note - too high values lead to long computation times and too low value result in matching errors and long computation times. If this is None then Ray Traversal is not used to generate initial guesses.
`min_cam_match`	Only particle that are recognized by this many of more cameras are considered real particles. particles seen by fewer than this are thrown away. Needs to be 2 or higher.
`max_err`	Maximum value of the RMS triangulation error in lab space coordinates (e.g. millimeters). Too low values of this parameter will lead to no particles that are found, but too high values will lead to erroneous results. As a general guide, this should be set much lower than the typical separation distance between particles in the experiment.
`save_name`	Path name used for saving the results; if `None` the results are not saved

3.5 Analyze disparity

As in every measurement system, 3D-PTV has its uncertainty; In 3D-PTV uncertainties are derived from various factors, starting from the quality of the calibration, through the quality of the image analysis performed (i.e. segmentation), and the use of the stereo-matching parameters. Our way of quantifying the results is by comparing the results of the end process (i.e. the particle positions in 3D) with the input to the system. Thus, in this step we are taking the positions of the particles we managed to calculate in the stereo-matching step, re-project these results on the various cameras, and then compare the initial results of our image analysis with the re-projection. The deviations from one another are called *disparities*. By analyzing statistics of the disparities

the uncertainty in our measurements can be estimated; furthermore, we can iteratively apply our analysis, and check the disparities in order to improve the results of our experiments.

Analyzing the disparities can be done in three easy ways using MyPTV. By running the workflow command `"analyze_disparities"`, a graphical user interphase (GUI) opens. The GUI allows plotting three kinds of figures which help in quantifying the results: a histogram for the disparities, a scatter-plot for the triangulation error and the disparity, and a visual figure showing the original blobs overlaid by the rep-projected particles. The parameters file parameters for this command are found in Table 8.

Table 8: The `params_file.yml` parameters for the **analyze disparity** step. All paths to files are relative to the `workflow.py` script.

Parameter	Description
`blob_files`	Names of files that hold the segmented particles for each camera, separated by commas; for example: `blobs_cam1, blobs_cam2, blobs_cam3`
`particle_filename`	Name of the file containing the stereo-matching results
`camera_names`	Names of the cameras separated by commas; for example: `cam1, cam2, cam3`
`max_err`	The maximal error used in the stereo-matching step (in physical units, e.g. millimeters)
`min_cam_match`	The minimum number of cameras that can be used to triangulate a particle, as used in the stereo-matching step

3.6 Tracking

The tracking step is used to link particles in the lab-space coordinates in time, thus forming the 3D trajectories. To initiate the tracking we use the workflow command `"tracking"`.

There are two tracking methods that have been implemented in MyPTV and the users can choose which one suits their needs best. The first algorithm is the "4 frames best estimate" tracking algorithm that was described in Ref. [6]. The second method, tentatively called here "multiframe" is a new adaptation of the best estimate algorithm. Its main advantage is that it is capable of linking particle trajectories over more than one frame, namely in cases where the particle was miss-identified in one of the images or if the was a glitch in its stereo matching. Its downside is that it might take longer time to process in particle-dense experiments and that it will not yield trajectories shorter than a given length tentatively called here the noise scale (`Ns`). The multiframe algorithm was not yet described in a paper, as it is a very recent addition. The description of the parameters used in the tracking step are given in Tab. 9.

3.7 Calibration with particles

After some trajectories have been obtained, we can refine the calibration of cameras by using this obtained data and re-run the calibration procedure with the measured particles. For more detailes see Sec. 5.2. To start calibration with particles enter the command `calibration_with_particles` in the command line following the workflow command. Once this is done, a calibration sequence is initiated that uses the trajectory data.

3.8 Smoothing

The smoothing step is used to smooth trajectories in time and to calculate the velocity and acceleration of particles. This is done by fitting polynomials over sliding windows of the trajectory where velocities and acceleration are calculated through a direct differentiation of the polynomial (see [7]).

3.9 Stitching

This step is used to re-track trajectories once velocities are known and to "stitch" broken trajectories following the algorithm of Ref. [8].

3.10 2D tracking guide

In addition to the 3D particle tracking capabilities, MyPTV can also be used to track particles in 2D, namely using only one camera view of the particles. The main advantages of 2D experiments are that they are easier and since only one camera is needed, and that the tracking is easier in 2D because there are no particle occlusions. Note that 2D particle tracking in MyPTV doesn't simply track the particle's pixel locations, since this might

Table 9: The `params_file.yml` parameters for the **tracking** step. All paths to files are relative to the `workflow.py` script.

Parameter	Description
`particles_file_name`	path name of the file which holds the 3D coordinates of particles, namely the results of the matching step
`method`	Here we choose the algorithm used for the tracking; can be either 'multiframe' of 'fourframe' (see Sec. 3.6).
`frame_start`	if `None` will start tracking from the first available frame. If an integer it will start the tracking from this given number.
`N_frames`	if `None` will iterate over particles in all frames; if an integer will only track particles in the first `N` frames of the particles file
`d_max`	the maximum translation in lab space coordinates
`dv_max`	the maximum allowable change in velocity in lab space coordinates per frame (e.g. mm/frame)
`max_dt`	The maximum allowed skip in frames over which a trajectory will be formed if using the multiframe method; e.g. if a particle is seen in frame 1, 2, 5, and 6, it will only be fully tracked if `max_dt`≥ 2. Note that the missing frames (3 and 4 in the example) are subsequently interpolated using a cubic polynomial.
`Ns`	A parameter of the multiframe method. It should be an odd integer or a list of odd integers. During tracking, links are formed only if a local signal to noise ratio does not exceed a certain level. The signal to noise ratio is calculated over windows of size `Ns`, with typical sizes of a few samples (e.g. 11). Note that the algorithm cannot build trajectories shorter than `Ns`. If a list is given then tracking is iterated using all the values in the list.
`NSR_threshold`	The NSR is a parameter of the multiframe algorithm used to assess the quality of a trajectories based on the notion that physical trajectories are smooth. The `NSR` takes values fraction positive values from 0 to infinity. A trajectory with $NSR = 0$ is considered really good while one with $NSR > 1$ is considered of quite low quality. Thus the `NSR_threshold` is the highest acceptable `NSR` value. Trajectories with `NSR` above the given threshold are discarded.
`mean_flow`	Adds a constant mean flow parameter that helps the tracking algorithm to track particles in flows with a constant drift; the format is [vx, vy, vz] in e.g. mm/frame.
`plot_candidate_graph`	If this is set to `True`, once tracking is done, it will show a graph of all the linking candidates. Red lines are links that were made and blue lines were candidates to be linked but they were rejected.
`save_name`	name of the file in which the results shall be saved; if `None` the results are not saved on the drive

Table 10: The `params_file.yml` parameters for the calibration with particles step.

Parameter	Description
`traj_filename`	the name of the file containing the trajectories from which the calibration points are taken
`camera`	an instance of the camera we wish to try and re-calibrate
`cam_number`	int, >= 1; the number (index) of the camera to be calibrated. For example, to calibrate camera2, this should be set to 2
`blobs_fname`	The name of the file that contains the segmented particles' data
`min_traj_len`	Only trajectories longer then this number will be used in the calibration
`max_point_number`	the maximum number of points that shall be taken to re-calibrate the camera. Note that too many points (say above 1000) might lead to long calculation times

give off errors due to camera aberrations or miss-alignment of the camera with the imaging plane. In practice this is done by making use of the optical model used in the 3D version, so this methodology is advantageous over simpler, pixel location, tracking approaches.

To track particles in 2D in MyPTV, we assume that all the particles shown in the image are located on a two-dimensional plane. In practice, this is done by setting a single z coordinate (given by the user) for all

Table 11: The `params_file.yml` parameters for the smoothing step. All paths to files are relative to the `workflow.py` script.

Parameter	Description
`trajectory_file`	file name of the trajectories file, namely the results of the tracking step
`window_size`	window size of the sliding polynomial (must be an odd number); if a trajectory is shorter than `window_size` but longer than `min_traj_length`, then we use a window the size of the trajectory length
`polynom_order`	degree of the polynomial used in the smoothing
`min_traj_length`	the length of the shortest trajectories to be smoothed; has to be larger than `min_traj_length`
`repetitions`	the smoothing operation will be repeated this many times over each trajectory
`save_name`	file name to save the results; if `None` the results will not be save on the disk

Table 12: The `params_file.yml` parameters for the `stitching` command. All paths to files are relative to the `workflow.py` script.

Parameter	Description
`trajectory_file`	name of the smoothed trajectory file used in the stitching
`max_time_separation`	maximum time separation over which to stitch broken trajectories
`max_distance`	maximum distance in the position-velocity space
`save_name`	file name to save the results; if `None` the results will not be save on the disk

particles and then tracking their motion over the x and y coordinates only (see Fig. 13).

The 2D particle tracking procedure requires following these steps:

1. Prepare a calibration folder as described in Sec. 3.1
2. Go through camera calibration as shown in Sec. 3.2
3. Perform particle segmentation of the particle images, as described in Sec. 3.3
4. Track the particles' coordinates using the workflow script using the `2D_tracking` command (see Sec. 3.1.1). The various parameters of the `params_file` associated with 2D tracking are given in Tab. 13
5. If needed, follow smoothing (Sec. 3.8) and stitching (Sec. 3.9) of the results

3.11 Manual Matching GUI

In certain applications a user might wish to probe the 3D positions of elements in the images manually. Examples might be, to test the quality of the calibration on regions outside of the calibration target, or to measure certain geometrical features of objects in the images. For that purpose MyPTV features a graphical user interface

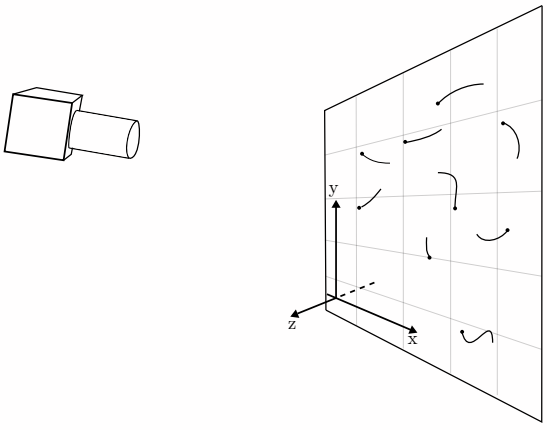


Figure 13: An example for experimental setup for 2D tracking. Note that the camera doesn't have to be perpendicular to the particles' plane and that the particles don't have to be at $z = 0$.

Table 13: The `params_file.yml` parameters for 2D tracking. All paths to files are relative to the `workflow.py` script.

Parameter	Description
`blob_file`	The name of the file that contains the segmentation results
`frame_start`	The number of the first frame from which we want to perform 2D tracking
`N_frames`	The number of frames to be analyzed; If `None`, then all the frames will be analyzed
`camera_name`	the name of the camera from which the images were taken (must be a calibrated camera)
`camera_resolution`	A tuple representing the camera resolution, for example, (1280, 1024)
`z_particles`	The value of the z coordinate of the plane on which the particles are found, e.g. 0.0
`d_max`	The maximum allowable particle translation between frame in lab-coordinates
`dv_max`	The maximum allowable change of velocity for the particles in lab-space coordinates per frame (e.g. mm/frame)
`save_name`	The name of the file used to save the results

dedicated to doing this easily. To run the GUI, use the `manual_matching` command of the `workflow.py` script.

To match a point we need to identify it from the different camera views. In the GUI we do this by clicking on the point we are after in the various images. See Fig. 14 for instructions.

Table 14: The `params_file.yml` parameters for the manual matching operation. All paths to files are relative to the `workflow.py` script.

Parameter	Description
`cameras`	a list that holds the names of the cameras used; for example [cam1, cam2, cam3]. Note that the camera files need to be in the same folder where we are running the workflow script
`images`	the paths of the images from which matching should be performed.

3.12 Fiber tracking

This module reconstructs the orientation of fibers in 3D space. After segmenting the fibers as described in Sec. 3.3.5, and tracking their centroids as described in Sec. 3.6, their orientations are reconstructed. There are two methods to recover orientations. The first method is with intersections of so-called *epipolar planes*. As shown in Fig. 15, an epipolar plane is defined by three points: two of them are contained on the 2D fiber orientation in image space, and the third one is the camera center. Given two epipolar planes for a camera pair, there exists a unique intersection between them, which defines the fiber orientation vector $\mathbf{p} = [p_x, p_y, p_z]$ in laboratory coordinates. The second, method works by minimizing the projection of the fiber on the image sensors, and it is based on the algorithm used in Ref. [9].

This principle of the plane intersection method is based on the same pin-hole model described in Eq. (1), and it is here briefly explained. Note that as it is based on the pinhole model, it is currently only available with the Tsai camera model in MyPTV. As shown in Fig. 15, for each camera image a synthetic point B is created from the 2D position and orientation of a segmented fiber. Then, the 3D directional vectors $\vec{r}_X - \vec{O}$ and $\vec{r}_B - \vec{O}$, pointing from the camera center through the segmented fiber center X and the synthetic point B, are calculated from Eq. (1). A plane projecting into 3D space is then obtained in each camera view by the vector triplet $\vec{r}_X - \vec{O}, \vec{r}_B - \vec{O}$, and $\vec{O}$. Finally, the intersection of two epipolar planes from a camera pair allows triangulating a line in 3D, which represents the fiber orientation.

The orientation calculation generates substantial errors when the fiber is parallel to the baseline of the camera pairs, i.e. the line connecting the two camera centers. This occurs when the relative angle between epipolar planes is small, and the intersection line is ill-defined. For this reason, the code outputs a NaN when the fiber is produced by epipolar planes with intersection angle less than the empirical threshold of 5°.

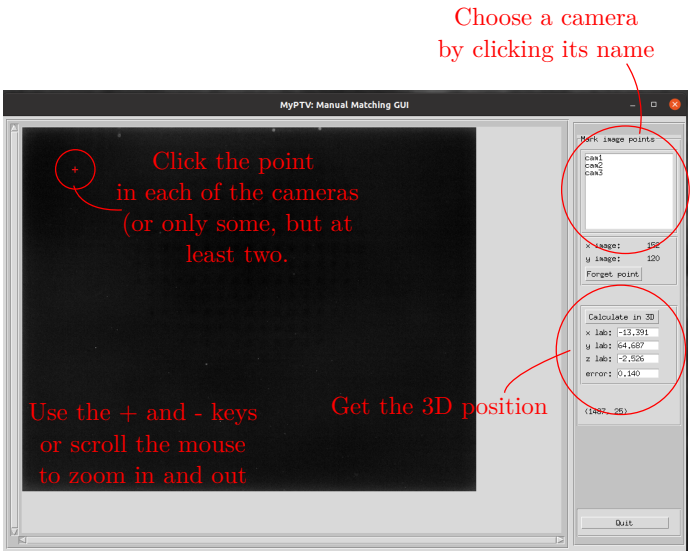


Figure 14: Instructions of how to use the manual matching GUI

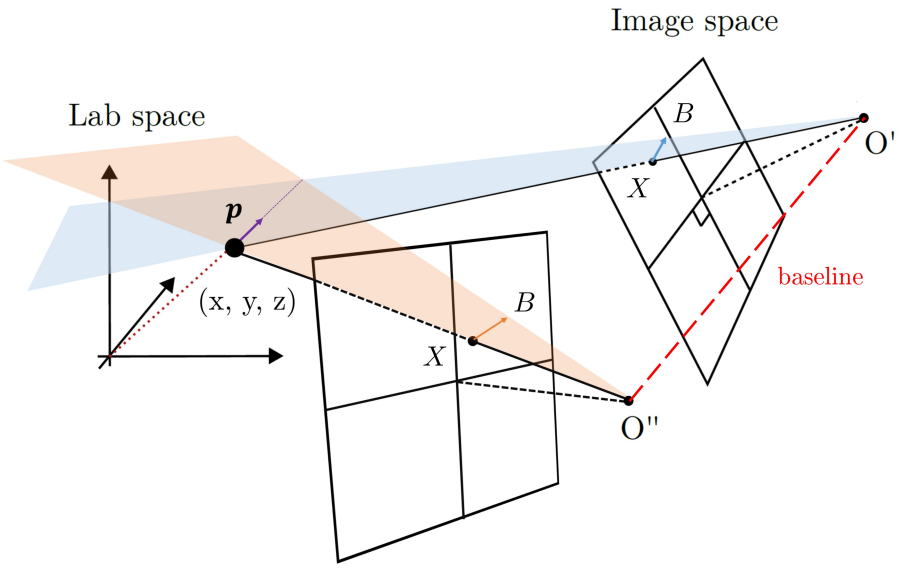


Figure 15: Schematic description of two epipolar planes spanned by the two camera centers. Their intersection generates the symmetry axis $\mathbf{p} = [p_x, p_y, p_z]$ of the fiber in laboratory coordinates. To define a plane, two points of a fiber in image space are here defined using the centroid, X, and the same point shifted with the 2D orientation vector, B. Figure adapted from Fabian Müller's Master thesis.

This module can be used with more than two cameras. As an example, when three cameras are imaging a fiber, the three possible fiber orientations are first generated from the possible camera pairs combination $(1-2)$, $(1-3)$ and $(2-3)$. The candidate $(1-2)$ is then projected on the image space of the remaining camera 3, and the *orientation error* is computed as the angle between the projected candidate and the segmented fiber orientation in that camera. The same is repeated for the other two candidates, and the final fiber orientation is the one belonging to the camera pair with the minimum orientation error.

A multi-camera orientation reconstruction is discarded when the orientation error exceeds the value set as `ori_lim`, which is set manually as a parameter. This limit depends on the fiber angular resolution in image space and must be estimated for each experiment as in the following example. Consider an experiment where the average length of a fiber in an image is 20 px. Then the angular resolution of imaged fibers is estimated as $\tan^{-1}(1/20) \approx 3°$. In this case, the `ori_lim` parameter can be set as `ori_lim` $= 5 \times (1 - \cos(3°)) \approx 0.005$, in this case to exceed 5 times the error given by the camera resolution.

To summarize, to reconstruct fiber orientations we use again the workflow script. We do this via the following steps:

1. In the segmentation step, make sure that the 'shape' parameter is set to 'fibers'.
2. Perform the segmentation, matching, tracking and smoothing, as normally done for spherical particles. You will note that once the segmentation is done, two blob files will be saved for each image: a regular one for the blob positions, and another one for the blob shapes.
3. To calculate the orientations in 3D, use the workflow script with the action '`fiber_orientations`'. Following this stage, a new results file will be saved with trajectories corresponding to the rotation angles of each particle. See Tab. 15 for the documentation on the information needed.

In the current version of MyPTV, the reconstruction works only with two or three cameras.

3.13 Fiber smoothing

This module is used to smooth both the fiber centroids and orientations and put them together in the same file. Analogous to particles, this is done by fitting polynomials over sliding windows of the trajectory where velocities and acceleration are calculated through a direct differentiation of the polynomial (see [7]). The data structure of the smoothed fiber data is similar to what is shown in Fig. 21, with the fiber orientation $\mathbf{p}$, rotation rate $d\mathbf{p}/dt$ and acceleration rate $d^2\mathbf{p}/dt^2$ components respectively from column 11 to 19, and with the frame number at column 20.

Table 15: The `params_file.yml` parameters for the `fiber_orientations` operation. All paths to files are relative to the `workflow.py` script.

Parameter	Description
`camera_names`	A list of the camera names, e.g. [cam1, cam2, cam3]. Note that the camera files need to be in the same folder where we are running the workflow script
`ori_lim`	A parameter which defines the threshold for maximum orientation error when more than two cameras are used. Only relevant for the `PlaneIntersect` method.
`method`	The method used to calculate fiber orientations; can be either `PlaneIntersect` or `MinProjection`. Note that if `PlaneIntersect` method is used then only up to three cameras can be used and they must be of Tsai model. The `MinProjection` method can take any number of cameras and of any camera model.
`blob_files`	The name of the files that hold the results of the **blob orientations**.
`trajectory_file`	The file name with the results of the tracking stage.
`save_name`	The file name that should be used to save the results of the orientation measurements.

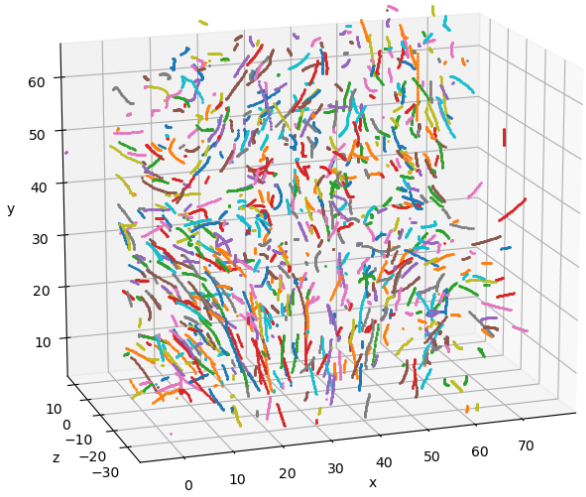


Figure 16: Trajectory segments plotted using the workflow `plot_trajectories` command. A 30 frames long segment is shown from a turbulent flow dataset taken from Ref. [10].

3.14 Fiber stitching

This module is used to stitch broken fiber trajectories using the same algorithm used for particles [8]. Once the centroids are stitched together, the broken orientations are reconstructed using a polynomial fit.

3.15 Plotting the results

Once the post processing is finished, we might want to plot trajectories in order to visualize the results. There are two basic options to do so using the workflow script: 1) A basic 3D rendering of measured trajectories can be established with the workflow command `plot_trajectories`. As example is shown in Fig. 16. The parameters for the plotting function are explained in Tab. 16. 2) Basic 3D animations of trajectories in a trajectory file can be prepared with the workflow command `animate_trajectories`. See Tab. 17 for the animation parameters.

Table 16: The `params_file.yml` parameters for the `plot_trajectories` operation. All paths to files are relative to the `workflow.py` script.

Parameter	Description
`file_name`	the path to the file that contains the trajectories; the file can be either in trajectories format or in smoothed trajectories format.
`min_lenth`	only trajectories that have more samples than this number will be plotted.
`write_trajID`	If True this will desplay the trajectory ID on top of them.
`t0` and `te`	Used to delineate the time range for which we plot the data. We only plot the samples in the time range starting at frame `t0` and ending at frame `te`. Set `t0=0` and `te=-1` to plot trajectories at all times available.

Table 17: The `params_file.yml` parameters for the `plot_trajectories` operation. All paths to files are relative to the `workflow.py` script.

Parameter	Description
`file_name`	the path to the file that contains the trajectories; the file can be either in trajectories format or in smoothed trajectories format.
`min_lenth`	only trajectories that have more samples than this number will be plotted.
`f_start` and `f_end`	Used to delineate the time range for which we plot the data. We only plot the samples in the time range starting at frame `t0` and ending at frame `te`. Set `f_start=None` and `f_end=None` to plot trajectories at all times available.
`fps`	This number is used as the frame per second for the animation.
`tail_length`	The length of trajectory segments shown at each time step.

4 Imaging module - `imaging_mod.py`

The imaging module is used to handle the translation from 2D image space coordinates to lab space coordinates and vice versa through the 3D model.

4.1 The `camera` object

An object that stores the camera external and internal parameters and handles the projections to and from image space and lab space. Inputs are:

1. `name` - string, name for the camera. This is the name used when saving and loading the camera parameters.
2. `resolution` - tuple (2), two integers for the camera number of pixels
3. `cal_points_fname` - string (optional), path to a file with calibration coordinates for the camera. The format fro the calibration point file is given in Section 4.3 (see Fig. 6a).

The important functionality options are:

1. `get_r(eta, zeta)` - Will solve eq. 1 for the orientation vector $\vec{b} = \vec{r} - \vec{O}$, given an input of pixel coordinates (η, ζ).
2. `projection(x)` - Will reverse solve equation (1) to find the image space coordinates (η, ζ), of an input 3D point, (x=$\vec{r}$).
3. `save(dir_path)` - Will save the camera parameters in a file called after the camera name in the given directory path, see Fig. 17.
4. `load(dir_path)` - Will load the camera parameters in a file called after the camera name in the given directory path, see Fig. 17.

After calibration we can save the camera parameters on the hard disc. The camera files have the structure shown in Fig. 17.

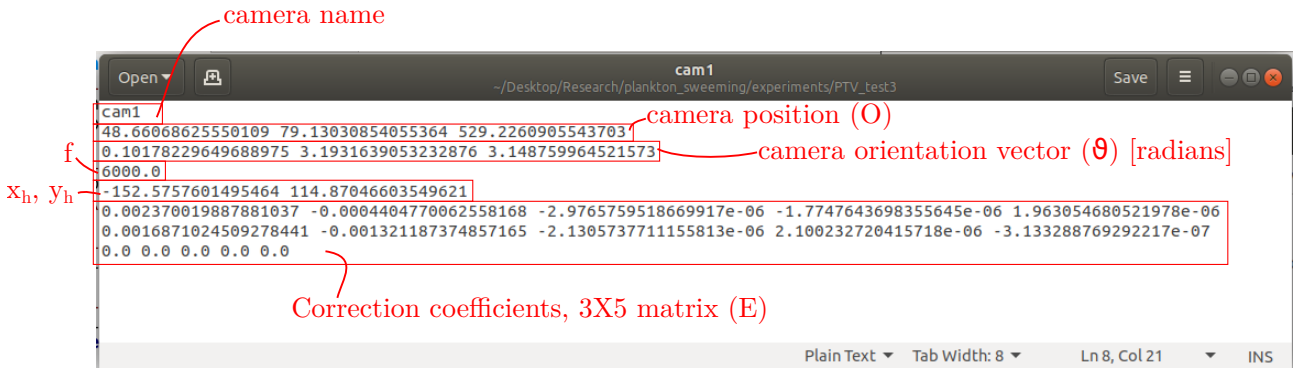


Figure 17: The structure of a camera file. The files are simple text files where each row corresponds to a specific parameter and the values in each row are separated by a white space.

4.2 The `imsys` object

An object that holds several camera instances and can be used to perform stereo-matching. The important functionalities are:

1. `stereo_match(coords, d_max)` - Takes as an input a dictionary with coordinates in image space from the several cameras and calculates the triangulation position.
   The coordinate dictionary has keys that are the camera number and the values which are the coordinates in each camera. d_max is maximum allowable distance for the triangulation.

4.3 The `Cal_image_coord` object

This is a class used for reading information given in the optional argument `cal_points_fname` of the `camera` class (Sec. 4.1). It is used internally and generally users will not have to deal with this. This class will read and interpret text files with tab separated valued, where the columns' meanings are: [x image space, y image space, x lab space, y lab space, z lab space], and each row is a single point of some known calibration target.

The input for this class is:

1. `fname` - String, the path to your calibration point file. The file is holds tab separated values with the meaning of: [x image, y image, x lab, y lab, z lab], see Fig. 6a.

5 Camera calibration - `calibrate_mod.py`

The `calibrate_mod.py` module, with the `calibrate` object, is used to find the camera calibration parameters. We calibrate each camera by taking an image of a *calibration target* - a body with markings of known coordinates in lab space - and search for the camera parameters that minimize the distance between the projection of the known points in image space and the image taken with the camera. In addition to that, after a calibration is made and trajectories have been obtained, the calibration might be refined by using the `calibrate_with_particles` object; this is particularly helpfull in cases where some of the cameras slightly moved in between the taking the calibration images and the recording of the tracer particles.

5.1 The `calibrate` object

Used to solve for the camera parameters given an input list of image space and lab space coordinates. The inputs are:

1. `camera` - An instance of a `camera` object which we would like to calibrate.
2. `lab_coords` - a list of lab space coordinates of some known calibration target.
3. `img_coords` - a list of image space coordinates that is ordered in accordance with the lab space coordinates.

The important functionalities are:

1. `searchCalibration(maxiter=5000, fix_f=True)` - When this is run, we use a nonlinear least squares search to find the camera parameters that minimize the cost function (item 3 below). This function is used to find the $\vec{O}$, $\vec{\theta}$, f, and x_h, y_h parameters (in case `fix_f=False`, it will not solve for f. `maxiter` is the maximum number of iterations allowed for the least squares search.
2. `fineCalibration(maxiter=500)` - This function will solve for the coefficients of the quadratic polynomial used for the nonlinear correction term ($[E]$).
3. `mean_squared_err` - This is our cost function, being the sum of distances between the image space coordinates and the projection of the given lab space coordinates.

To find an optimal calibration solution, we might need to run each function several times, and run the coarse and fine calibrations one after the other until a satisfactory solution is obtained. Once it is obtained, we should keep in mind to save the results using the `save` functionality of the `camera` object.

5.2 The `calibrate_with_particles` object

A class used to refine the calibration using particles data. In short, after the primary clibration is done, matching and tracking can be used to obtain trajectories from the experimental data. Assuming that this resulted in trajectories that were successfully measured, we can leverage the trajectories obtained to minimize further the calibration error. The refinement is done by assuming that the positions of particles along the resolved trajectories are "true" positions in lab-space, and thus, the blobs corresponding to these particles are used in a calibration sequence to minimize the calibration error. The assumption is that longer trajectories with low triangulation error are considered more reliable as compared to shorter trajectories, so we use only "long enough" trajectories in this process.

The inputs are:

1. `traj_filename` - the name of the file containing the trajectories from which the calibration points are taken.
2. `camera` - an instance of the camera we wish to try and re-calibrate
3. `cam_number` - int, $>= 1$; the number (index) of the camera to be calibrated. For example, to calibrate camera2, this should be set to 2.
4. `blobs_fname` - The name of the file that contains the segmented particles' data.
5. `min_traj_len` - Only trajectories longer then this number will be used in the calibration

6. `max_point_number` - the maximum number of points that shall be taken to re-calibrate the camera. Note that too many points (say above 1000) might lead to long calculation times.

The important functionalities are:

1. `get_calibrate_instance` - This function returns an instance of the `calibrate` object with the calibration points taken from the trajectory file. We then use this object to refine the calibration using the regular procedure (Sec. 5.1).

5.3 The `gui_final_cal.py` file

The `gui_final_cal.py` script is used to launch a `tkinter` based graphical user interface that is utilized to run various functionalities of the calibration process. The GUI is shown in Fig. 4, and the various buttons are explained in Tab. 3.

5.4 The `gui_initial_cal.py` file

The `gui_initial_cal.py` script is used to launch a `tkinter` based graphical user interface that is utilized to run various functionalities of the calibration process. The GUI is shown in Fig. 3 and it is explained in section 3.2.3.

6 Particle segmentation - `segmentation_mod.py`

This module handles the image analysis part of MyPTV, taking in raw camera images containing particles and output their image space coordinates. For the segmentation we first blur the image to remove salt and pepper noise, then we highlight particles using a local mean subtraction around each pixel, and then use a global threshold to mark foreground and pixels. Finally, the connected foreground pixels are considered to be particles, and we estimate the blob's center using a brightness weighted average of blob pixels.

6.1 The `particle_segmentation` object

Used to segment particles in a given image. This class is used internally to iterate over frames in a single folder by the `loop_segmentation` class. However, it is recommended to check the segmentation parameters manually using this `particle_segmentation` over several images in order to tune the particle searching parameters. The inputs are:

1. `image` - the image for segmentation
2. `threshold=10` - the global filter's threshold brightness value, so pixels with brightness higher than this number are considered foreground
3. `mask=1.0` - A mask matrix can be used to specify rigions of interest within the image
4. `sigma=None` - If float, this is taken as the standard deviation of a Gaussian blurring filter; if it is None, no bluring is performed
5. `median=None` - If integer this is taken as the window size for a median noise removal filter; if None, no median filter is performed
6. `local_filter=None` - Parameters for a local mean subtraction. If it's an interger it is taken as the window size; is it None, no mean subtraction is performed.
7. a bunch of threshold pixel sizes and mass in all directions and in area.
8. `method` - String. The name of the method used for segmentation (either 'labeling' or 'dilation', see Sec. 3.3). Default is 'labeling'.
9. `particle_size=3` - The particle size used in the dilation method.

The important funcitonalities are:

1. `get_blobs` - Will return a list of blob centers, their box size and their area.
2. `plot_blobs()` - Uses matplotlib to plot the results of the segmentation. A very useful functionality in the testing of segmentation parameters!

3. `save_results(fname)` - Will save the segmented particles in a text file. The file is arranged in six columns with the following attributes: (x center position, y center position, x size, y size, area, image number), see Fig. 18.

6.2 The `loop_segmentation` object

An object used for looping over images in a given directory to segment particles and save the results in a file. The inputs are nearly identical to those of `particle_segmentation`.

important functionalities are:

1. `segment_folder_images()` - Will loop over the images in the given directory and segment particles according to the given parameters
2. `save_results(fname)` - Will save the segmented particles in a text file. The file is arranged in six columns with the following attributes: (x center position, y center position, x size, y size, area, image number), see Fig. 18.

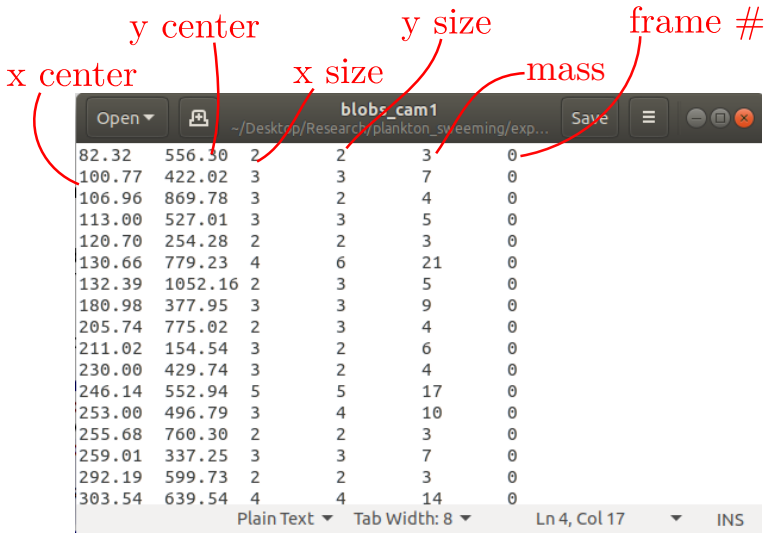


Figure 18: An example of a text file holding the segmentation resuls and the description of the different columns.

7 Particle matching - `particle_matching_mod.py`

The module used to link particles in the different images through stereo matching and estimating their 3D positions.

7.1 The `matching_with_marching_particles_algorithm` object

One of the main issue in this process is that stereo matching all possible candidates is an NP hard problem, so to track numerous particles in each frame we have to choose which particles are likely to produce a 3D particle coordinate. To solve this we are using an algorithm called particle marching. The principles are as follows:

1) Given an initial guess for a particle position in lab space coordinates, we can project it onto camera space coordinates of our camera system. The results of the projection can be denoted [(x1, y1), ..., (xn, yn)], where n is the number of cameras in our imaging system.

2) The segmentation results from the experiment will be called blobs. For each point (xi, yi) from (1), we choose the blob that has the smallest distance to it. If the nearest neighbor blob in a certain camera was already used up previously, then we take the 2nd nearest blob (or the 3rd,...). We denote the nearest neighbor blobs we found as [(xNN1, yNN1), ..., (xNNn, yNNn)].

3) Taking the NN blobs, we try to see if the epipolar lines associated with them cross each other. If more than a threshold number of lines cross, then the point at which they crossed is considered the position of a real particle. Otherwise, the initial guess is deemed bad and we dump it.

The thing that is left it to have a method for choosing initial guesses for the lab space particles. We are using for that a combination of three methods -

1) If particles were found in the previous frame (say frame number i-1), then these positions are used as initial guesses for frame i. This allows to have continuous trajectories already from the matching process and really speeds up the process. It is also possible to do the same process going backwards, i.e. using particles from frame i+1 as the initial guesses. From this concept comes the name of this algorithm.

2) Random guesses across the measurement volume. This is not so efficient but it does work and it allows starting up the marching process.

3) Using the ray traversal algorithm over pairs of cameras to get initial guesses. This works quite good, however when there are many blobs in each frame it can take up a lot of time. This also allows starting up the marching process.

7.2 The `match_blob_files` object (Legacy)

This is the object that we use to get triangulated particles results from the segmented blob files (a file as the one in Fig. 18 for each camera). For each frame it first runs the first algorithm using time information, and only then uses the Ray traversal algorithm on the blobs that were not successfully connected. The inputs are:

1. `blob_fnames` - a list of the (srting) file names containing the segmented blob data. The list has to be sorted according the order of cameras in the `img_system`.
2. `img_system` - an instance of the `img_system` class with the calibrated cameras.
3. `RIO` - A nested list of 3X2 elements. The first holds the minimum and maximum values of x coordinates, the second is same for y, and the third for z coordinates.
4. `voxel_size` - the side length of voxel cubes used in the ray traversal algorithm. Given in lab space coordinates (e.g. mm). Note - a too large voxel size will result in high computational times due a high number of candidates, while a too small voxel size might lead to erroneous intersection of rays, leading to matching errors. Thus, this parameter should be optimized.
5. `max_blob_dist` - the largest distance for which blobs are considered neighbours in the image space coordinates (namely, the largest permissible blob displacement in pixels).
6. `max_err=None` - Maximum acceptable uncertainty in particle position. If None, (defult), than no bound is used.
7. `reverse_eta_zeta=False` - Should be false if the eta and zeta coordinates need to be in reverse order so as to match the calibration. This may be needed if the calibration data points were given where the x and y coordinates are transposed (as happens, e.g., if using matplotlib.pyplot.imshow).

The important functionalities are:

1. `get_particles()` - Use this to match blobs into particles in 3D.
2. `save_results(fname)` - Save the results in a text file. The format has 4 + number of cameras columns separated by tabs: (x, y, z, [N columns corresponding to the blob number in each camera] , frame number, see Fig. 19).

7.3 The `matching` object (Legacy)

This object is the "engine" used to match particles using the Ray Traversal algorithm. In practice we run the relevant functions: `get_voxel_dictionary()` → `list_candidates()` → `get_particles()`, and after that the results are held in the attribute `matched_particles`.

7.4 The `matching_using_time` object (Legacy)

This object performs the matching of blobs using the 2D tracking heuristic. In principle, it is given a list of blobs that were successfully used to form 3D particles in the previous frame. Then, for each of the given blobs it searches for nearest neighbours in the current frame, and stereo-matches those blobs that were found (using the `.triangulate_candidates()` method).

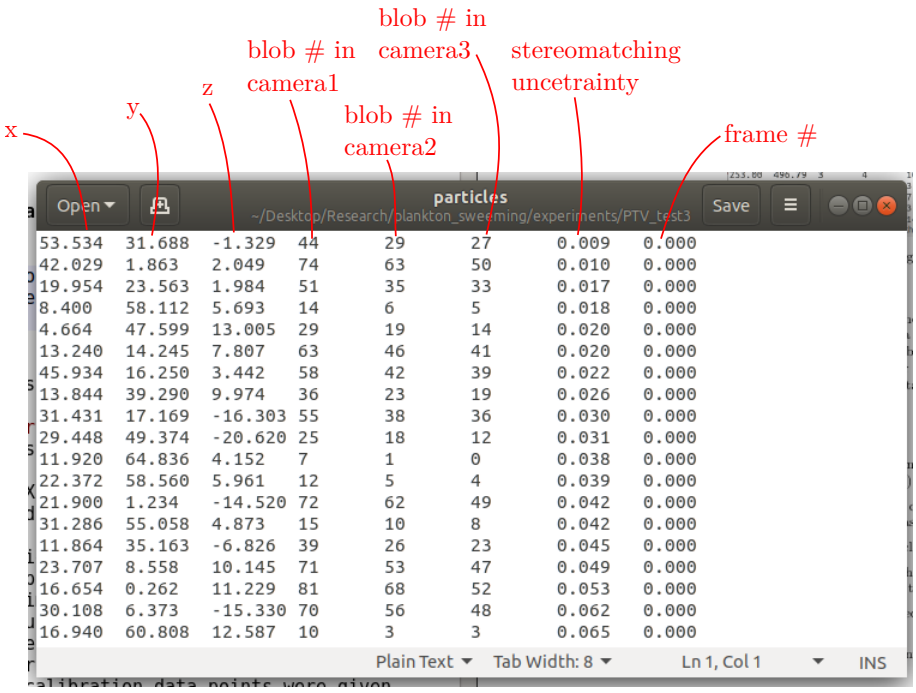


Figure 19: An example of a text file holding the triangulated particles' resuls and the description of the different columns. In this example there were three cameras. The blob number columns give the index of the blobs corresponding to any particle at the this specific frame number; a value of -1 in one of the rows means that no blob was used to stereo-match the particle in this row for this particular camera.

7.5 The `initiate_time_matching` object (Legacy)

This object is used to initiate the time searching algorithm on the first frame. It goes over the blobs at the first frame and searches for blobs that have nearest neighbours at the second frame. Those that have neighbours are used in a first run of the Ray Traversal algorithm, thus they are given priority in the search.

8 Tracking in 3D - `tracking_mod.py`

This is the module that is used to track particles in 3D. There are currently three tracking methods implemented, nearest neighbour, two-frame, and four-frame, see Ref. [6]. Users are welcome to choose their preferred method and use it.

8.1 The `tracker_four_frames` object

An object used to perform tracking through the 4-frame best estimate method [6]. Input:

1. `fname` - a string name of a particle file (e.g. Fig. 19
2. `mean_flow=0.0` - either zero (default) of a numpy array of the mean flow vector, in units of the calibrations spatial units per frame (e.g. mm per frame). The mean flow is assumed not to change in space and time.
3. `d_max_=1e10` - maximum allowable translation between two frames for the nearest neighbour search, after subtracting the mean flow.
4. `dv_max_=1e10` - maximum allowable change in velocity for the two-frame velocity projection search. The radius around the projection is therefore dv_max/dt (where dt = 1 frame^{-1})
5. `store_candidates = False` - Boolean indicator. If it is true, then the tracker stores all the possible links that correspond to the tracking thresholds. In this case we can then plot a particle linking graph to analyze the tracking quality. If it is False, the tracker does not keep this information.

The important functionality is:

1. `track_all_frames()` - Will track particles through all the frames.
2. `return_connected_particles()` - Will retun the list of trajectories that were established.

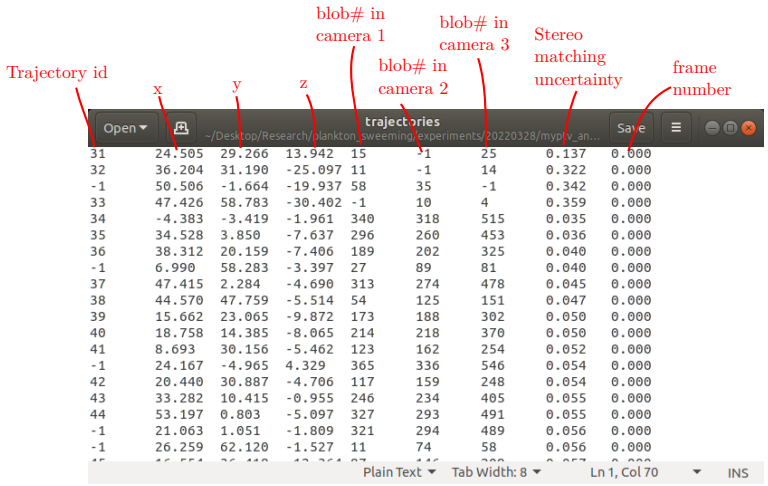


Figure 20: Example of a trajectory file and the column definitions. For Trajectory id being a non-negative integer, rows with the same Trajectory id correspond to the same trajectory; rows with Trajectory id being -1 are samples that could not be linked with the given tracking parameters.

3. `save_results(fname)` - Will save the results on the hard drive. The results are saved in a text file, where each row is a sample of a trajectory. The columns are specified as follows: [trajectory number, x, y, z, frame number], see Fig 20.
4. `plot_candidate_graph()` - If the candidate links were stored, this method will plot a graph that shows the links that were made and the candidate links that could have been made but were eventually not made due to duplicate links for the same particle.

8.2 The `tracker_two_frames` object

An object used for tracking through the 2-frame method. The description is the same as in Section 8.1

8.3 The `tracker_nearest_neighbour` object

An object used for tracking through the nearest neighbour method. The description is the same as in Section 8.1

9 Trajectory smoothing - `traj_smoothing_mod.py`

This module is used to smooth trajectories and to calculate the velocity and acceleration of the particles. For the smoothing we are using the polynomial fitting method proposed and used in Refs. [7, 11]. In short, each component of the particle's position is fitted with a series of polynomials with a sliding window of fixed and the derivatives are calculated by analytically differentiating the polynomial. The end result is a new file with smoothed trajectories. However note that we smooth and calculate velocities and accelerations only for trajectories longer than the window size for the smoothing (a user decided parameter).

9.1 The `smooth_trajectories` object

A class used to smooth trajectories in a list of trajectories. Due to the smoothing we also calculate the velocity and acceleration of the trajectories. The input trajectory list structure is the same as the files produced by the classes in `tracking_mod.py`.

Note - only trajectories whose length is larger than the window size will be smoothed and saved. Shorter trajectories are saved with zero velocity and accelerations.

The inputs are:

1. `traj_list` - a list of samples organized as trajectories. This should have the same data structure used in the saving function of the tracking algorithms (see Section 8.1).
2. `window` - The window size used in the sliding polynomial fitting.
3. `polyorder` - The order of the polynomial used in the fitting.

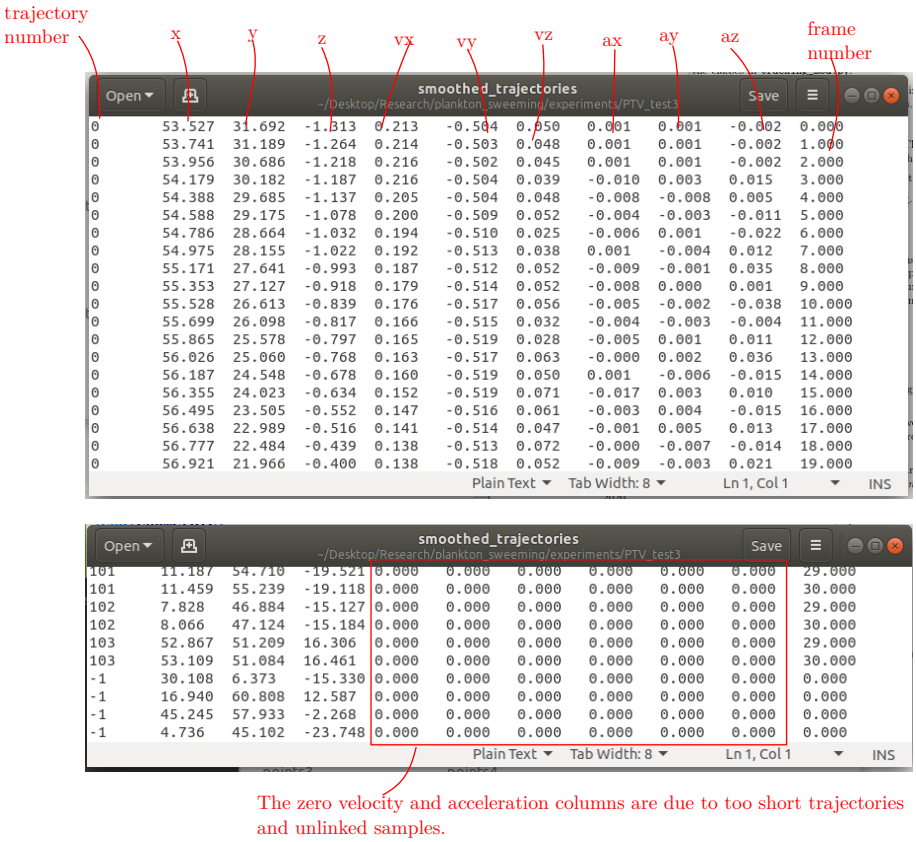


Figure 21: Example file holding the results of smoothed trajectories, and the description for each column. Note also the unsmoothed samples at the bottom of the file.

The important functionality is:

1. `smooth()` - performs the actual smoothing
2. `save_results(fname)` - Saves the results on the hard drive using the given (string) file name. The resulting file is a text file such that each row is a sample of a trajectory, and with 11 columns. The columns have the following meaning: [traj number, x, y, z, v_x, v_y, v_z, a_x, a_y, a_z, frame number], where v_i and a_i denote components of the velocity and acceleration vectors respectively, see Fig. 21.

10 Trajectory stitching - `traj_stitching_mod.py`

This module applies the algorithm by Ref. [8] to connect trajectories that were broken along the process by tracking the trajectories again in the position–velocity space. We also extend this by interpolating the missing samples using a 3rd order polynomial, that is fitted to the existing 4 data points at the tips of the broken trajectories. This module is applied after the trajectory smoothing step.

10.1 The `traj_stitching` object

This object performs the stitching process. Inputs:

1. `traj_list` - the list of smoothed trajectory, given as a Numpy array of shape (N,11), where N is the number of samples. The format is the same as the format generated after the smoothing process.
2. `Ts` - The maximum number of broken samples allowed in the connection.
3. `dm` - The maximum distance between the trajectory for which connections are made.

The important functionalities are:

1. `stitch_trajectories()` - Will search for candidates and stitch the best fitting candidates. Run this function to perform the stitching. After running the new trajectory list is held as the attribute `new_traj_list`.
2. `save_results(fname)` - Will save the stitched trajectories in a text file with a given file name. The format for the saved file is the same as the one used in the smoothing trajectories (Fig. 21).

References

[1] M. Virant and T. Dracos. 3D PTV and its application on Lagrangian motion. *Measurement Science and Technology*, 8:1552–1593, 1997.

[2] H. G. Maas, D. Gruen, and D. Papantoniou. Particle tracking velocimetry in three-dimensional flows Part I: Photogrammetric determination of particle coordinates. *Experiments in Fluid*, 15:133–146, 1993.

[3] Andreas Geiger, Frank Moosmann, Ömer Car, and Bernhard Schuster. Automatic camera and range sensor calibration using a single shot. In *2012 IEEE International Conference on Robotics and Automation*, pages 3936–3943. IEEE, 2012.

[4] Zhengyou Zhang. A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11):1330–1334, 2000.

[5] M. Bourgoin and S. G. Huisman. Using ray-traversal for 3D particle matching in the context of particle tracking velocimetry in fluid mechanics. *Review of scientific instruments*, 91(8):085105, 2020.

[6] N. T. Ouellette, H. Xu, and E. Bodenschatz. A quantitative study of three-dimensional Lagrangian particle tracking algorithms. *Experiments in Fluids*, 40(2):301–313, 2006.

[7] B. Lüthi, A. Tsinober, and W. Kinzelbach. Lagrangian measurement of vorticity dynamics in turbulent flow. *Journal of Fluid mechanics*, 528:87–118, 2005.

[8] H. Xu. Tracking Lagrangian trajectories in position–velocity space. *Measurement Science and Technology*, 19(7):075105, 2008.

[9] S Bounoua, G Bouchet, and Gautier Verhille. Tumbling of inertial fibers in turbulence. *Physical review letters*, 121(12):124502, 2018.

[10] R. Shnapp, S. Brizzolara, M. M. Neamtu Halic, A. Gambino, and M. Holzner. Universal alignment in turbulent pair dispersion. *Nature Communications*, 14(1), 2023.

[11] R. Shnapp, E. Shapira, D. Peri, Y. Bohbot-Raviv, E. Fattal, and A. Liberzon. Extended 3D-PTV for direct measurements of Lagrangian statistics of canopy turbulence in a wind tunnel. *Scientific reports*, 9(1):1–13, 2019.